LEÇON PRATIQUE · JAVA 21 / 25 · MISE À JOUR NOVEMBRE 2026

Énumérations Java de A à Z

Créer des `enum` complètes, les comparer, les trier,
et construire une hiérarchie — explicite, ou automatique « par le nom ».

Public	Développeuse / développeur Java qui connaît les bases (classes, collections) et veut maîtriser les énumérations pour de bon : au-delà de `enum Couleur { ROUGE, VERT }`.
Prérequis	Un JDK 21 ou 25 (`javac -version` répond), un terminal, et savoir compiler un fichier `.java`. Aucune connaissance de Maven ou Gradle n'est nécessaire : tous les exemples sont en ligne de commande.
Durée	≈ 3 h : lecture 1 h 30 · reproduction des exemples 45 min · exercices 45 min.
Objectif final	Savoir répondre sans hésiter à : « dans quel ordre mes constantes sont-elles triées ? », « comment comparer deux énumérations ? », « comment représenter une hiérarchie alors qu'un enum ne peut pas hériter ? », et « existe-t-il un tri automatique par le nom ? ».

enum · values · valueOf · ordinal | Comparable · Comparator | EnumSet · EnumMap | TreeSet · TreeMap | switch exhaustif · patterns | JLS §8.9

La question de départ, et sa réponse courte

« Peut-on trier des énumérations automatiquement par leur nom ? » **Oui — mais ce n'est pas l'ordre par défaut.** Un `enum` est `Comparable` : son ordre naturel est l'**ordre de déclaration** (via `ordinal()`), pas l'ordre alphabétique. Pour trier par nom, il faut demander explicitement `Comparator.comparing(MonEnum::name)` (chapitre 4.3). Et l'ordre alphabétique pur réserve deux surprises : `ETAPE_10` passe avant `ETAPE_2`, et les accents se classent après le `Z` (chapitre 4.4).

« Existe-t-il une hiérarchie automatique entre constantes ? » **Non.** Java n'offre aucun héritage entre constantes : un enum ne peut pas étendre une classe, et deux constantes sont toujours sœurs. La hiérarchie se construit *à la main*, avec un champ `parent` (chapitre 5). Mais l'ordre de déclaration fournit gratuitement une hiérarchie *linéaire* — c'est ce que le chapitre 5.5 détaille, avec les conventions de nommage qui la rendent fiable.

Sources principales : *Java Language Specification* SE 25, §8.9 « Enum Classes » · Javadoc `java.lang.Enum`, `java.util.EnumSet`, `java.util.EnumMap`, `java.util.Comparator`, `java.lang.Comparable`, `java.util.stream.Collectors`, `java.text.Collator` · code source du JDK 25 (`src.zip`, `java.lang.Enum`) pour les citations d'implémentation. Toutes les commandes et toutes les sorties de ce document ont été exécutées sur Debian 13 avec Temurin 25.0.5, en novembre 2026.

Sommaire

1 Ce qu'est réellement une énumération en Java 3
- 1.1 Déclarer : la syntaxe minimale et ce qu'elle cache 3
- 1.2 Ce que le compilateur génère (et pourquoi c'est une classe finale) 3
- 1.3 `name()`, `ordinal()`, `toString()` : trois choses différentes 5
- 1.4 Identité, immuabilité, sérialisation : les garanties 7

2 Créer des énumérations complètes 9
- 2.1 Constructeur, champs et méthodes 9
- 2.2 Un comportement par constante (corps de constante) 10
- 2.3 Implémenter une interface : l'énumération polymorphe 12
- 2.4 Méthodes statiques utiles : `fromCode`, cache, `values()` 13
- 2.5 Cinq erreurs de compilation qu'on rencontre vraiment 15

3 Comparer des énumérations 17
- 3.1 `==`, `equals`, `compareTo` : lequel et quand 17
- 3.2 `Comparable` : l'ordre naturel est l'ordre de déclaration 17
- 3.3 Comparer par un attribut, pas par `ordinal()` 18
- 3.4 Les pièges de `compareTo` (null, type, ordinal figé) 19

4 Trier des énumérations 21
- 4.1 Les cinq façons de trier, et ce qu'elles donnent 21
- 4.2 Trier par attribut, inverser, combiner 22
- 4.3 Trier par le nom : `comparing(MonEnum::name)` 23
- 4.4 Le tri « naturel » : nombres, accents, casse 23
- 4.5 Structures déjà triées : `EnumSet`, `EnumMap`, `TreeSet`, `TreeMap` 25
- 4.6 Trier des objets qui contiennent une énumération 27

5 Construire une hiérarchie 30
- 5.1 Pourquoi un enum ne peut pas hériter 30
- 5.2 La hiérarchie explicite : un champ `parent` 30
- 5.3 Parcourir : profondeur, racines, sous-arbres, enfants 33
- 5.4 Quand `sealed` + `record` remplacent l'enum 34
- 5.5 La hiérarchie « automatique » par le nom : ce qui existe vraiment 35

6 `switch` exhaustif et pattern matching 40
- 6.1 `switch` sur enum : la forme moderne 40
- 6.2 `null`, `default`, et l'exhaustivité 41
- 6.3 Patterns : `case ROUGE`, `case null`, `when` 41
- 6.4 Ajouter une constante : ce qui casse, ce qui ne casse pas 42

7 Cas pratiques vérifiés 44
- 7.1 Un cycle de vie avec transitions autorisées 44
- 7.2 Regrouper et compter avec `EnumMap` 45
- 7.3 Persister : toujours `name()`, jamais `ordinal()` 45
- 7.4 Mesures : `values()`, `EnumSet`, `EnumMap` 46
- 7.5 Tester une énumération avec JUnit 5 48

8 Erreurs et pièges : 12 cas réels 50

9 Fiche de synthèse et aide-mémoire 54

10 Exercices corrigés 57

11 Sources et pour aller plus loin 66

1 Ce qu'est réellement une énumération en Java

> Une énumération n'est pas une « liste de constantes ». C'est une **classe** — finale, avec un nombre fixé d'instances créées au chargement — qui hérite de `java.lang.Enum`. Comprendre cela règle d'un coup la moitié des questions de tri, de comparaison et de hiérarchie de ce document.

Question	Réponse vérifiée dans le JDK 25
Est-ce une classe ?	Oui. `enum Priorite` compile en une classe `final` qui étend `java.lang.Enum<Priorite>` et implémente `Comparable` et `Serializable`.
Combien d'instances ?	Exactement une par constante, créées à l'initialisation de la classe. Ce sont des singletons par construction.
Peut-on en créer une autre ?	Non : constructeur privé, pas de `clone()`, et la réflexion refuse explicitement l'instanciation (chapitre 1.4).
Peut-elle hériter d'une classe ?	Non, jamais : elle hérite déjà de `Enum`. Elle peut en revanche implémenter des interfaces (chapitre 2.3).
Deux constantes peuvent-elles être parentes ?	Non. Deux constantes d'un même enum sont toujours au même niveau, sœurs. La hiérarchie se code à la main (chapitre 5).
Quel est son ordre naturel ?	L'ordre de **déclaration**, via `ordinal()`. Pas l'ordre alphabétique (chapitre 3.2).

1.1 Déclarer : la syntaxe minimale et ce qu'elle cache

FICHIER — **PRIORITE.JAVA**, L'ÉNUMÉRATION LA PLUS SIMPLE POSSIBLE

```
public enum Priorite {
    BASSE, NORMALE, HAUTE, URGENTE
}
```

Quatre lignes, et pourtant tout est déjà là : quatre instances uniques, un ordre figé (`BASSE < NORMALE < HAUTE < URGENTE`), une méthode `values()`, une méthode `valueOf(String)`, et l'aptitude à être utilisée dans un `switch`. Le point-virgule final est facultatif *tant qu'on n'ajoute ni champ ni méthode* — dès qu'on en ajoute, il devient obligatoire (section 2.5, erreur 2).

> **La convention de nommage, et pourquoi elle compte pour le tri**
>
> Les constantes s'écrivent `EN_MAJUSCULES_AVEC_UNDERSCORES`. Ce n'est pas qu'une mode : c'est ce qui rend le tri par nom lisible (`BASSE, HAUTE, NORMALE, URGENTE`), et c'est ce qui permet de reconstruire une constante depuis un identifiant externe avec `valueOf`. Le JLS précise que le nom d'une constante est exactement l'identifiant déclaré ; il ne peut donc contenir ni espace, ni tiret, ni accent — mais `ÉTÉ` est techniquement un identifiant Java valide, avec les surprises de tri que l'on verra au chapitre 4.4.

1.2 Ce que le compilateur génère (et pourquoi c'est une classe finale)

Plutôt que de le croire, vérifions-le avec l'outil `javap` livré dans le JDK. C'est la commande à retenir pour inspecter n'importe quelle énumération, y compris celle d'une bibliothèque.

COMMANDES

```
javac -d out src/main/java/fr/nice/enums/Priorite.java
javap -p -cp out fr.nice.enums.Priorite
```

SORTIE RÉELLE

```
Compiled from "Priorite.java"
public final class fr.nice.enums.Priorite extends java.lang.Enum<fr.nice.enums.Priorite> {
  public static final fr.nice.enums.Priorite BASSE;
  public static final fr.nice.enums.Priorite NORMALE;
  public static final fr.nice.enums.Priorite HAUTE;
  public static final fr.nice.enums.Priorite URGENTE;
  private final int poids;
  private final java.lang.String libelle;
  private static final fr.nice.enums.Priorite[] $VALUES;
  public static fr.nice.enums.Priorite[] values();
  public static fr.nice.enums.Priorite valueOf(java.lang.String);
  private fr.nice.enums.Priorite(int, java.lang.String);
  public int poids();
  public java.lang.String libelle();
  public boolean auMoins(fr.nice.enums.Priorite);
  private static fr.nice.enums.Priorite[] $values();
  static {};
}

Compiled from "Priorite.java"
public final class fr.nice.enums.Priorite extends java.lang.Enum<fr.nice.enums.Priorite> {
  public static final fr.nice.enums.Priorite BASSE;
  public static final fr.nice.enums.Priorite NORMALE;
  public static final fr.nice.enums.Priorite HAUTE;
  public static final fr.nice.enums.Priorite URGENTE;
  public static fr.nice.enums.Priorite[] values();
  public static fr.nice.enums.Priorite valueOf(java.lang.String);
  public fr.nice.enums.Priorite[] values$suspend();
  private fr.nice.enums.Priorite();
}
```

Trois enseignements immédiats. **Un** : chaque constante est un champ `public static final` du type de l'enum — c'est pourquoi `Priorite.HAUTE` est une constante de compilation, et pourquoi il n'y a pas besoin de `new`. **Deux** : la classe est `final` et hérite de `Enum` ; impossible d'en hériter, et c'est exactement ce qui interdit une hiérarchie par héritage. **Trois** : le tableau `$VALUES` et le constructeur privé apparaissent — nous y revenons juste après. Notez le constructeur privé : `Priorite(int, java.lang.String)` — *deux* paramètres, comme dans la source. La section 2.1 montre ce que devient cette signature vue de l'intérieur de la JVM, avec les champs `name` et `ordinal` ajoutés par le compilateur.

COMMANDE — OÙ EST STOCKÉ L'ORDRE, ET COMMENT `VALUES()` TRAVAILLE

```
javap -c -cp out fr.nice.enums.Priorite | sed -n '1,20p'
```

SORTIE RÉELLE

```
public final class fr.nice.enums.Priorite extends java.lang.Enum<fr.nice.enums.Priorite> {
  public static final fr.nice.enums.Priorite BASSE;
  public static final fr.nice.enums.Priorite NORMALE;
  public static final fr.nice.enums.Priorite HAUTE;
  public static final fr.nice.enums.Priorite URGENTE;

  public static fr.nice.enums.Priorite[] values();
    Code:
         0: getstatic     #16     // Field $VALUES:[Lfr/nice/enums/Priorite;
         3: invokevirtual #20     // Method "[Lfr/nice/enums/Priorite;".clone:()Ljava/lang/Object;
         6: checkcast     #21     // class "[Lfr/nice/enums/Priorite;"
         9: areturn

  public static fr.nice.enums.Priorite valueOf(java.lang.String);
    Code:
         0: ldc           #1      // class fr.nice.enums/Priorite
         2: aload_0
         3: invokestatic  #25     // Method java/lang/Enum.valueOf:(Ljava/lang/Class;Ljava/lang/
String;)Ljava/lang/Enum;
         6: checkcast     #1      // class fr.nice.enums/Priorite
         9: areturn
```

Deux choses s'y lisent. **Le `clone()` de `values()`** : c'est la preuve, au niveau du bytecode, que chaque appel réalloue un tableau (chapitre 2.4). **Le bloc `static {}`** (non affiché ici, mais présent dans la classe) contient la suite des `new Priorite("BASSE", 0)`, `new Priorite("NORMALE", 1)`… : l'ordre de déclaration y est gravé sous forme d'un compteur. C'est de là que vient `ordinal()`, et c'est pourquoi réordonner les constantes change le comportement de tout code qui s'appuie sur `compareTo`.

> **Conséquence pratique n°1 : l'ordre de déclaration est une API**
>
> Déclarer `URGENTE` avant `BASSE` inverse tous les tris naturels de votre application. Ce n'est pas un détail cosmétique : si un tri de liste ou une comparaison `switch` en dépend, réordonner les constantes est un changement de comportement, à documenter et à tester. La règle : déclarez dans l'ordre du domaine (du plus petit au plus grand, du début à la fin), et ne réordonnez jamais sans raison explicite.

1.3 `name()`, `ordinal()`, `toString()` : trois choses différentes

Méthode	Retour	Ce qu'elle renvoie	Peut-on la redéfinir ?
`name()`	`String`	Le nom exact déclaré : `"HAUTE"`.	**Non** — `final`. Stable d'une version à l'autre.
`ordinal()`	`int`	La position, à partir de 0 : `BASSE`=0, `URGENTE`=3.	**Non** — `final`. Instable si on réordonne ou insère.
`toString()`	`String`	Par défaut, la même chose que `name()`.	**Oui** — c'est fait pour l'affichage.

PROGRAMME — `EXTRAITDEBASE.JAVA`

```
Priorite p = Priorite.HAUTE;
System.out.println("name()      = " + p.name());
System.out.println("ordinal()   = " + p.ordinal());
System.out.println("toString()  = " + p);
System.out.println("values()    = " + java.util.Arrays.toString(Priorite.values()));
System.out.println("valueOf     = " + Priorite.valueOf("NORMALE"));
System.out.println("compareTo   = HAUTE vs BASSE -> " + Priorite.HAUTE.compareTo(Priorite.BASSE));
System.out.println("identity    = " + (Priorite.HAUTE == Priorite.valueOf("HAUTE")));
```

SORTIE RÉELLE

```
name()      = HAUTE
ordinal()   = 2
toString()  = HAUTE
values()    = [BASSE, NORMALE, HAUTE, URGENTE]
valueOf     = NORMALE
compareTo   = HAUTE vs BASSE -> 2
identity    = true
```

> **La règle à retenir : `name()` pour la machine, `toString()` pour l'humain**
>
> La Javadoc de `Enum` est explicite : « *Most programmers should use the `toString` method in preference to this one, as the `toString` method may return a more user-friendly name. This method [`name()`] is designed primarily for use in specialized situations where correctness depends on getting the exact name, which will not vary from release to release.* » Autrement dit : **`name()`** pour la persistance, les fichiers de configuration, les API et les tests ; **`toString()`** pour les messages affichés à l'utilisateur.

Voici les deux ensemble, sur une énumération qui redéfinit `toString()` pour l'affichage :

FICHIER — UNE ÉNUMÉRATION « AFFICHABLE »

```
public enum Jour {
    LUNDI("lundi"), MARDI("mardi"), MERCREDI("mercredi"), JEUDI("jeudi"),
    VENDREDI("vendredi"), SAMEDI("samedi"), DIMANCHE("dimanche");

    private final String libelle;

    Jour(String libelle) {
        this.libelle = libelle;
    }

    @Override
    public String toString() {
        return libelle;
    }
}
```

RÉSULTAT

```
System.out.println(Jour.VENDREDI);          # toString()
System.out.println(Jour.VENDREDI.name());   # name()
System.out.println(Jour.valueOf("VENDREDI"));
```

SORTIE RÉELLE

```
vendredi
VENDREDI
vendredi
```

> **Le piège de la redéfinition de `toString()`**
>
> Après cette redéfinition, `Jour.valueOf("vendredi")` **échoue** : `valueOf` se base sur `name()`, pas sur `toString()`. Et un `EnumSet.toString()` affichera `[lundi, mardi]` — lisible, mais inutilisable pour reconstruire les constantes. Si vous redéfinissez `toString()`, prévoyez toujours une méthode dédiée pour revenir à la constante (section 2.4).

1.4 Identité, immuabilité, sérialisation : les garanties

Une énumération Java offre quatre garanties que peu de langages donnent, et qui expliquent pourquoi on l'utilise massivement pour les états, les types et les codes :

Garantie	Conséquence	Preuve (vérifiée)
Une instance par constante	`==` suffit pour comparer ; `equals` est identique à `==`.	`Priorite.HAUTE == Priorite.valueOf("HAUTE")` → `true`
Pas de clonage	Aucun risque de copie modifiée : `clone()` lève une exception.	`Enum.clone()` : `throw new CloneNotSupportedException()`
Pas d'instanciation par réflexion	Impossible de fabriquer une cinquième `Priorite`.	`IllegalArgumentException : Cannot reflectively create enum objects`
Sérialisation par le nom	Un enum désérialisé *est* la constante du singleton, pas une copie.	`désérialisé == constante ? true`

PROGRAMME — VÉRIFICATION DES QUATRE GARANTIES

```
import java.io.*;
import java.lang.reflect.*;
import java.util.*;

enum Mois { JANVIER, FEVRIER, MARS }

// 1. identité
System.out.println("identité : " + (Mois.FEVRIER == Mois.valueOf("FEVRIER")));

// 2. réflexion refusée
Constructor<?> c = Mois.class.getDeclaredConstructors()[0];
c.setAccessible(true);
try { c.newInstance("X", 9, 9); }
catch (IllegalArgumentException e) { System.out.println("réflexion : " + e.getMessage()); }

// 3. sérialisation
ByteArrayOutputStream bo = new ByteArrayOutputStream();
try (ObjectOutputStream oo = new ObjectOutputStream(bo)) { oo.writeObject(Mois.FEVRIER); }
System.out.println("octets sérialisés : " + bo.toByteArray().length);
try (ObjectInputStream oi = new ObjectInputStream(new ByteArrayInputStream(bo.toByteArray()))) {
    Object o = oi.readObject();
    System.out.println("désérialisé == constante : " + (o == Mois.FEVRIER));
}
```

SORTIE RÉELLE

```
identité : true
réflexion : Cannot reflectively create enum objects
octets sérialisés : 69
désérialisé == constante : true
```

Ce que 69 octets signifient

La sérialisation d'un enum n'écrit pas les champs : elle écrit le nom du type et le nom de la constante. C'est pour cela que la taille ne dépend pas du nombre d'attributs (ajoutez dix champs, la taille ne bougera presque pas) et que la constante retrouvée est le singleton. Corollaire important : **le contrat de sérialisation est le nom**. Renommer une constante casse la lecture des données existantes — voir chapitre 7.3.

Ce que la JLS dit exactement

Le chapitre 8.9 de la *Java Language Specification* définit la classe d'énumération : elle est `final` sauf si elle déclare au moins une constante avec un corps (auquel cas elle est scellée implicitement et les constantes sont ses sous-classes anonymes — chapitre 2.2). Elle hérite de `Enum<E>`, implémente `Comparable<E>` et `Serializable`, et son constructeur est privé. Tout ce document découle de cette section.

Ce que ces 69 octets ne disent pas : l'ordre et l'identité

La taille constant s'explique autrement : deux champs seulement (`name`, `ordinal`) et pas de table de hachage. Corollaire peu connu, et pourtant décisif pour la suite de cette leçon : ce qui n'est *pas* dans l'objet, c'est l'ordre de déclaration. Il n'est stocké nulle part dans la constante — il n'existe que parce que le compilateur numérote les constantes dans l'ordre du fichier source et que `values()` reconstruit le tableau dans cet ordre.

PROGRAMME — OÙ VIT RÉELLEMENT L'ORDRE ?

```
List<String> noms = Arrays.stream(Priorite.class.getDeclaredFields())
                          .map(Field::getName).toList();
System.out.println("noms des champs : " + noms);
System.out.println("ordre de values(): " + Arrays.toString(Priorite.values()));
System.out.println("ordinal de HAUTE : " + Priorite.HAUTE.ordinal());
System.out.println("values()[2]      : " + Priorite.values()[2]);
```

SORTIE RÉELLE

```
noms des champs : [BASSE, NORMALE, HAUTE, URGENTE, poids, libelle, $VALUES]
ordre de values(): [BASSE, NORMALE, HAUTE, URGENTE]
ordinal de HAUTE : 2
values()[2]      : HAUTE
```

Deux conséquences pratiques, que cette leçon exploitera sans arrêt. **D'abord**, l'ordre de déclaration est un contrat implicite : déplacer une ligne dans le fichier change `ordinal()`, change `compareTo`, change l'ordre d'itération de `EnumSet` — et casse les fichiers qui auraient stocké l'`ordinal` (chapitre 7). **Ensuite**, il n'y a aucun lien automatique entre le nom d'une constante et sa position : c'est précisément ce qui rend impossible une « hiérarchie automatique par le nom » (chapitre 5).

La règle d'or qui découle de cette section

Traitez l'ordre de déclaration comme une décision documentée : du moins prioritaire au plus prioritaire, du général au particulier, du début à la fin d'un cycle de vie. Si vous devez un jour insérer une constante *au milieu*, vous savez que vous ne pouvez le faire qu'en fin de liste — sinon vous modifiez l'ordre naturel de tout le reste.

2 Créer des énumérations complètes

> Une constante qui porte des données, un comportement propre, une interface, des méthodes statiques de conversion : c'est ce qui distingue une énumération utilitaire d'une énumération de projet. Cinq formes à connaître, dans l'ordre de complexité croissante.

2.1 Constructeur, champs et méthodes

FICHIER — `PRIORITE.JAVA` AVEC POIDS ET LIBELLÉ

```
public enum Priorite {
    BASSE(1, "basse"),
    NORMALE(2, "normale"),
    HAUTE(3, "haute"),
    URGENTE(4, "urgente");

    private final int poids;
    private final String libelle;

    Priorite(int poids, String libelle) {   // implicitement privé
        this.poids = poids;
        this.libelle = libelle;
    }

    public int poids() { return poids; }

    public String libelle() { return libelle; }

    /** Vrai si cette priorité est au moins aussi élevée que {@code autre}. */
    public boolean auMoins(Priorite autre) {
        return this.compareTo(autre) >= 0;
    }
}
```

Trois règles à respecter, sans exception :

- **Le point-virgule après la dernière constante est obligatoire** dès qu'un membre suit. L'oubli donne un message peu clair (`';' expected`), traité en section 2.5.
- **Les champs sont `final`**. Une énumération immuable est la seule qui préserve les garanties du chapitre 1.4. Un champ mutable partagé entre constantes est un bug d'état global.
- **Le constructeur est privé** — implicitement, et il ne peut pas être rendu public. Vous n'appelez jamais `new Priorite(…)` : les seuls appels sont ceux du compilateur, dans le bloc statique de la classe.

VÉRIFICATION

```
javap -p -cp out fr.nice.enums.Priorite
```

SORTIE RÉELLE (EXTRAIT)

```
public final class fr.nice.enums.Priorite extends java.lang.Enum<fr.nice.enums.Priorite> {
  public static final fr.nice.enums.Priorite BASSE;
  public static final fr.nice.enums.Priorite NORMALE;
  public static final fr.nice.enums.Priorite HAUTE;
  public static final fr.nice.enums.Priorite URGENTE;
  private final int poids;
  private final String libelle;
  public static fr.nice.enums.Priorite[] values();
  public static fr.nice.enums.Priorite valueOf(java.lang.String);
  public int poids();
  public java.lang.String libelle();
  public boolean auMoins(fr.nice.enums.Priorite);
  private fr.nice.enums.Priorite(java.lang.String, int, int, java.lang.String);
}
```

Pourquoi le constructeur a-t-il quatre paramètres ?

`Priorite(String, int, int, String)` : les deux premiers sont ajoutés par le compilateur — le **nom** et l'**ordinal** — puis viennent vos deux arguments `poids` et `libelle`. C'est la démonstration concrète que `name()` et `ordinal()` sont stockés dans l'instance, pas calculés à la volée. C'est aussi pourquoi on ne peut pas définir un constructeur d'énumération qui prendrait un `String` en premier paramètre sans ambiguïté de lecture.

2.2 Un comportement par constante (corps de constante)

Quand chaque constante doit se comporter différemment, deux approches existent. La première est un `switch` dans une méthode — simple, mais le compilateur ne vous préviendra pas si vous ajoutez une constante. La seconde est le **corps de constante** : chaque constante fournit sa propre implémentation, et l'ajout d'une constante devient impossible à oublier.

FICHIER — `TAILLE.JAVA`, CORPS DE CONSTANTE ET MÉTHODE ABSTRAITE

```
public enum Taille {
    XS(1) {
        @Override public String categorie() { return "petite"; }
    },
    S(2) {
        @Override public String categorie() { return "petite"; }
    },
    M(3) {
        @Override public String categorie() { return "moyenne"; }
    },
    L(4) {
        @Override public String categorie() { return "grande"; }
    };

    private final int rang;

    Taille(int rang) { this.rang = rang; }

    public int rang() { return rang; }

    public abstract String categorie();
}
```

RÉSULTAT

```
for (Taille t : Taille.values()) {
    System.out.println(t + " rang=" + t.rang() + " catégorie=" + t.categorie());
}
```

SORTIE RÉELLE

```
XS rang=1 catégorie=petite
S rang=2 catégorie=petite
M rang=3 catégorie=moyenne
L rang=4 catégorie=grande
```

Le prix à payer : `Taille.XS.getClass()` n'est plus `Taille`

Dès qu'une constante a un corps, le compilateur crée une **sous-classe anonyme** pour elle. `Taille.XS.getClass()` renvoie `fr.nice.enums.Taille$1`, et non `Taille`. Conséquences : `getClass() == Taille.class` est faux, `getClass().isEnum()` est faux aussi (c'est la *superclasse* qui est un enum), et certaines bibliothèques qui font des tests d'égalité de classe peuvent se comporter bizarrement. Utilisez `getDeclaringClass()`, qui renvoie toujours `Taille` :

VÉRIFICATION

```
System.out.println(Taille.XS.getClass());            // fr.nice.enums.Taille$1
System.out.println(Taille.XS.getDeclaringClass());   // class fr.nice.enums.Taille
System.out.println(Taille.XS.getClass().isEnum());   // false
System.out.println(Niveau.INFO.getClass());          // fr.nice.enums.Niveau (pas de corps)
```

SORTIE RÉELLE

```
class fr.nice.enums.Taille$1
class fr.nice.enums.Taille
false
class fr.nice.enums.Niveau
```

Faut-il alors éviter les corps de constante ? Non — c'est la bonne façon d'implémenter un comportement qui varie par constante. Mais préférez un `switch` exhaustif quand la logique est courte et lisible, et réservez les corps de constante aux cas où chaque constante porte une vraie implémentation (une opération arithmétique, un algorithme de calcul, un format d'export).

2.3 Implémenter une interface : l'énumération polymorphe

FICHIERS — UNE INTERFACE, PUIS UNE ÉNUMÉRATION QUI L'IMPLÉMENTE

```
public interface Etiquette {
    String etiquette(String message);
}

public enum Niveau implements Etiquette {
    INFO("[i]"),
    AVERTISSEMENT("[!]"),
    ERREUR("[X]");

    private final String prefixe;

    Niveau(String prefixe) { this.prefixe = prefixe; }

    @Override
    public String etiquette(String message) {
        return prefixe + " " + message;
    }
}
```

RÉSULTAT

```
for (Niveau n : Niveau.values()) {
    System.out.println(n.etiquette("service indisponible"));
}
```

SORTIE RÉELLE

```
[i] service indisponible
[!] service indisponible
[X] service indisponible
```

L'intérêt est considérable : une méthode qui accepte un `Etiquette` fonctionne avec `Niveau` *et* avec n'importe quelle classe qui implémente l'interface. C'est la façon propre d'utiliser une énumération sans coupler tout le code à son type précis — et c'est aussi le seul « héritage » possible pour un enum.

FICHIER — `STATUT.JAVA` : UN CYCLE DE VIE, AVEC `SWITCH` EXHAUSTIF

```
public enum Statut {
    OUVERT,
    EN_COURS,
    RESOLU,
    FERME;

    public boolean peutAllerVers(Statut suivant) {
        return switch (this) {
            case OUVERT    -> suivant == EN_COURS || suivant == FERME;
            case EN_COURS  -> suivant == RESOLU   || suivant == FERME;
            case RESOLU    -> suivant == EN_COURS || suivant == FERME;
            case FERME     -> false;
        };
    }
}
```

RÉSULTAT

```
for (Statut s : Statut.values()) {
    System.out.println(s + " peut aller vers " +
        java.util.Arrays.stream(Statut.values()).filter(s::peutAllerVers).toList());
}
```

SORTIE RÉELLE

```
OUVERT peut aller vers [EN_COURS, FERME]
EN_COURS peut aller vers [RESOLU, FERME]
RESOLU peut aller vers [EN_COURS, FERME]
FERME peut aller vers []
```

Pourquoi `switch` et pas une table ?

Un `switch` *expression* (avec `->`) sur un enum est vérifié à la compilation : si vous ajoutez `ANNULE` à `Statut`, le code ne compile plus tant que vous n'avez pas traité le cas. C'est une protection gratuite, et c'est la raison de préférer cette forme à une `Map` de transitions construite à la main (qui, elle, échouerait silencieusement à l'exécution). Le chapitre 6.4 en fait la démonstration.

2.4 Méthodes statiques utiles : `fromCode`, cache, `values()`

Deux conversions reviennent sans arrêt dans un vrai projet : retrouver une constante depuis un code externe (base de données, JSON, paramètre d'URL) et énumérer les valeurs sans réallouer un tableau à chaque appel.

FICHIER — ÉNUMÉRATION AVEC RECHERCHE PAR CODE ET CACHE INTERNE

```
import java.util.Map;
import java.util.function.Function;
import java.util.stream.Collectors;
import java.util.stream.Stream;

public enum Devise {
    EUR("978", "€"),
    USD("840", "$"),
    GBP("826", "£"),
    CHF("756", "CHF");

    private final String codeNumerique;
    private final String symbole;

    Devise(String codeNumerique, String symbole) {
        this.codeNumerique = codeNumerique;
        this.symbole = symbole;
    }

    public String codeNumerique() { return codeNumerique; }

    public String symbole() { return symbole; }

    // Cache construit une seule fois, à l'initialisation de la classe
    private static final Map<String, Devise> PAR_CODE =
            Stream.of(values()).collect(Collectors.toUnmodifiableMap(Devise::codeNumerique,
Function.identity()));

    // Conversion tolérante : casse et espaces ignorés
    public static Devise fromCode(String code) {
        Devise d = PAR_CODE.get(code);
        if (d == null) {
            throw new IllegalArgumentException("Code devise inconnu : " + code);
        }
        return d;
    }

    // Variante pour les entrées utilisateur (nom, insensible à la casse)
    public static Devise fromNom(String nom) {
        return Stream.of(values())
                .filter(v -> v.name().equalsIgnoreCase(nom.trim()))
                .findFirst()
                .orElseThrow(() -> new IllegalArgumentException("Devise inconnue : " + nom));
    }
}
```

RÉSULTAT

```
System.out.println(Devise.fromCode("840"));
System.out.println(Devise.fromNom(" eur "));
try { Devise.fromCode("999"); }
catch (IllegalArgumentException e) { System.out.println("erreur : " + e.getMessage()); }
```

SORTIE RÉELLE

```
USD
EUR
erreur : Code devise inconnu : 999
```

> **Le piège de l'initialisation statique circulaire**
>
> Le champ `PAR_CODE` appelle `values()` : il doit donc être déclaré **après** les constantes, et la JVM garantit que les constantes sont créées avant toute autre initialisation statique de la classe. Si vous placez ce champ avant les constantes, vous obtenez `IllegalArgumentException: null` ou un tableau vide — un classique du genre. Règle simple : constantes d'abord, champs statiques ensuite.

`values()` est coûteux : pourquoi et comment l'éviter

`values()` ne renvoie pas un tableau partagé : il en **clone un nouveau à chaque appel**, précisément pour empêcher un appelant de modifier les constantes via le tableau. Deux appels successifs renvoient donc deux objets différents :

VÉRIFICATION

```
System.out.println("values() == values() : " + (Priorite.values() == Priorite.values()));
System.out.println("EnumSet.allOf        : " + EnumSet.allOf(Priorite.class));
```

SORTIE RÉELLE

```
values() == values() : false
EnumSet.allOf        : [BASSE, NORMALE, HAUTE, URGENTE]
```

Dans une boucle exécutée des millions de fois, cela finit par se voir. La mesure ci-dessous compare deux millions d'itérations avec et sans cache, sur la même JVM (Temurin 25.0.5, Debian 13) :

PROGRAMME DE MESURE (ÉCHAUFFEMENT PUIS MEILLEUR TEMPS SUR 5 PASSES)

```
// variante A : values() à chaque itération
for (int i = 0; i < N; i++) { Priorite[] v = Priorite.values(); s += v[i % v.length].ordinal(); }

// variante B : tableau récupéré une seule fois
Priorite[] v = Priorite.values();
for (int i = 0; i < N; i++) { s += v[i % v.length].ordinal(); }
```

SORTIE RÉELLE (N = 2 000 000)

```
2M x values()           : 6807 us
2M x tableau en cache   :  959 us
```

Facteur 7. La correction est triviale : sortez l'appel de la boucle, ou utilisez `EnumSet.allOf(MaClasse.class)` qui est une vue immuable et non un tableau à recopier.

2.5 Cinq erreurs de compilation qu'on rencontre vraiment

Ces cinq erreurs ont été provoquées avec `javac -Xlint:all` sur le JDK 25. Les messages sont recopiés tels quels.

Erreur	Message réel	Correctif
`enum E extends Base`	`error: '{' expected` puis `error: enum constant expected here`	Un enum ne peut pas étendre une classe. Utilisez une interface (2.3) ou un champ `parent` (5.2).
Membre avant la première constante	`error: enum constant expected here`	Les constantes viennent toujours en premier, avant tout champ ou méthode.
`implements Comparable<E>` + `compareTo`	`error: compareTo(E) in E cannot override compareTo(E) in Enum — overridden method is final`	C'est inutile : `Enum` l'implémente déjà, en `final`. Changez d'ordre avec un `Comparator` (chapitre 3.3).
Référence avant déclaration	`error: illegal forward reference`	Une constante ne peut référencer qu'une constante déclarée avant elle. Réordonnez, ou passez par une méthode.
`enum E<T>`	`error: enums cannot be generic`	Un enum ne peut pas être générique. Utilisez une interface générique implémentée par l'enum.

REPRODUCTION — L'ERREUR D'HÉRITAGE, MESSAGE EXACT

```
public class T { static class Base {} enum E extends Base { A, B } }
```

SORTIE RÉELLE

```
T.java:1: error: '{' expected
public class T { static class Base {} enum E extends Base { A, B } }
                                         ^
T.java:1: error: enum constant expected here
public class T { static class Base {} enum E extends Base { A, B } }
                                           ^
2 errors
```

Le message le plus trompeur : `illegal forward reference`

Écrire `A(B), B(A)` semble naturel pour deux constantes qui se référencent mutuellement. C'est refusé : la JVM doit créer `A` avant `B`, donc `B` n'existe pas encore quand `A` la référence. La solution est de ne pas stocker la référence directe : stockez un nom, un code, ou un identifiant, et résolvez la relation dans une méthode (c'est exactement ce que fait le champ `parent` du chapitre 5.2 pour les hiérarchies — à une différence près : une hiérarchie est *acyclique*, donc chaque enfant référence un parent déjà déclaré, ce qui est autorisé).

BONNE NOUVELLE : LE CAS DE LA HIÉRARCHIE EST LÉGAL

```
public enum Categorie {
    INCIDENT(null),        // racine
    RESEAU(INCIDENT),      // référence une constante déjà déclarée : OK
    DNS(RESEAU);           // OK aussi
}
```

Retenez la règle : **une constante ne peut référencer que ce qui est déclaré avant elle**. Une hiérarchie se déclare donc toujours « du haut vers le bas » — ce qui est aussi l'ordre naturel de lecture d'un arbre.

3 Comparer des énumérations

> Trois notions se confondent souvent : l'**identité** (« est-ce la même constante ? »), l'**égalité** (même chose, mais via `equals`) et l'**ordre** (« laquelle vient avant ? »). Une énumération Java fournit les trois, gratuitement, mais avec des règles précises qu'il faut connaître pour ne pas écrire de code faux.

3.1 `==`, `equals`, `compareTo` : lequel et quand

Opération	Réponse	Quand l'utiliser
`a == b`	Identité d'instance	**Toujours** pour comparer deux constantes entre elles. C'est plus rapide, plus lisible, et impossible à casser par une redéfinition d'`equals` — il n'y en a pas.
`a.equals(b)`	Identique à `==`	Uniquement quand le second terme est de type `Object` (dans un `Set`, un `Map`, un `Objects.equals`). Utile aussi pour la symétrie de code générique.
`a.compareTo(b)`	`< 0`, `0`, `> 0`	Pour **ordonner** : tris, `TreeSet`, `TreeMap`, bornes. Jamais pour tester l'égalité — utilisez `==`.
`Objects.equals(a, b)`	`true` même si l'un est `null`	Quand une des deux valeurs peut être `null` (champ optionnel, retour de méthode).

PROGRAMME — LES QUATRE OPÉRATIONS SUR LES MÊMES VALEURS

```
System.out.println("identité  : " + (Priorite.HAUTE == Priorite.valueOf("HAUTE")));
System.out.println("equals    : " + Priorite.HAUTE.equals(Priorite.valueOf("HAUTE")));
System.out.println("compareTo : HAUTE vs BASSE -> " + Priorite.HAUTE.compareTo(Priorite.BASSE));
System.out.println("compareTo : HAUTE vs HAUTE  -> " + Priorite.HAUTE.compareTo(Priorite.HAUTE));
System.out.println("Objects.equals(null, BASSE) : " + java.util.Objects.equals(null,
Priorite.BASSE));
```

SORTIE RÉELLE

```
identité  : true
equals    : true
compareTo : HAUTE vs BASSE -> 2
compareTo : HAUTE vs HAUTE  -> 0
Objects.equals(null, BASSE) : false
```

> **La règle en une ligne**
>
> `==` pour savoir si c'est la même constante, **`compareTo`** pour savoir laquelle vient avant. `equals` n'apporte rien de plus qu'`==` ; si vous le rencontrez dans du code, c'est souvent un reste de génération automatique d'IDE — ce n'est pas faux, seulement inutile.

3.2 `Comparable` : l'ordre naturel est l'ordre de déclaration

Chaque énumération implémente `Comparable<E>`. L'implémentation est dans `java.lang.Enum`, et le code source du JDK 25 ne laisse aucun doute :

EXTRAIT DU CODE SOURCE DU JDK 25 — `JAVA.BASE/JAVA/LANG/ENUM.JAVA`, LIGNES 215-221

```
public final int compareTo(E o) {
    Enum<?> other = o;
    Enum<E> self = this;
    if (self.getClass() != other.getClass() && // optimization
        self.getDeclaringClass() != other.getDeclaringClass())
        throw new ClassCastException();
    return self.ordinal - other.ordinal;
}
```

Deux conséquences, tirées directement de ces lignes :

- **La comparaison porte sur `ordinal()`**, donc sur la position dans la déclaration. `BASSE(0) < NORMALE(1) < HAUTE(2) < URGENTE(3)`.
- **La méthode est `final`** : on ne peut pas la redéfinir pour changer l'ordre naturel. La seule façon de changer l'ordre est de réordonner les constantes, ou d'utiliser un `Comparator` (section 3.3).

PROGRAMME — L'ORDRE NATUREL EN ACTION

```
System.out.println("ordinal BASSE   = " + Priorite.BASSE.ordinal());
System.out.println("ordinal URGENTE = " + Priorite.URGENTE.ordinal());
System.out.println("HAUTE.compareTo(BASSE)    = " + Priorite.HAUTE.compareTo(Priorite.BASSE));
System.out.println("BASSE.compareTo(URGENTE)  = " + Priorite.BASSE.compareTo(Priorite.URGENTE));
System.out.println("sorted()                  = " + List.of(Priorite.URGENTE, Priorite.BASSE,
Priorite.NORMALE).stream().sorted().toList());
```

SORTIE RÉELLE

```
ordinal BASSE   = 0
ordinal URGENTE = 3
HAUTE.compareTo(BASSE)    = 2
BASSE.compareTo(URGENTE)  = -3
sorted()                  = [BASSE, NORMALE, URGENTE]
```

Documentez l'ordre, il fait partie du contrat public

Puisque l'ordre naturel dépend de la déclaration, tout code appelant qui trie, compare ou itère sur votre énumération *dépend de votre ordre de déclaration*. Si votre énumération est une API publique, écrivez-le dans la Javadoc : « L'ordre naturel est du moins prioritaire au plus prioritaire ». Et si le domaine n'a pas d'ordre naturel évident (une liste de pays, des codes de couleur), dites-le aussi : cela évitera qu'un collègue construise une logique sur un ordre arbitraire.

3.3 Comparer par un attribut, pas par `ordinal()`

Il existe une tentation : trier selon un champ métier (`poids`, `code`, `rang`) en utilisant `ordinal()`, sous prétexte que « dans notre cas, l'ordre de déclaration correspond au poids ». C'est fragile. Comparez explicitement l'attribut :

PROGRAMME — TROIS COMPARATEURS QUI NE DONNENT PAS LE MÊME RÉSULTAT

```
List<Priorite> melange = new ArrayList<>(List.of(Priorite.URGENTE, Priorite.BASSE, Priorite.HAUTE,
Priorite.NORMALE));

List<Priorite> parOrdinal = new ArrayList<>(melange);
parOrdinal.sort(null);                                    // ordre naturel = ordinal

List<Priorite> parPoids = new ArrayList<>(melange);
parPoids.sort(Comparator.comparingInt(Priorite::poids));  // explicite

List<Priorite> parNom = new ArrayList<>(melange);
parNom.sort(Comparator.comparing(Priorite::name));        // alphabétique

System.out.println("ordinal : " + parOrdinal);
System.out.println("poids   : " + parPoids);
System.out.println("nom     : " + parNom);
System.out.println("ordinal == poids ? " + parOrdinal.equals(parPoids));
```

SORTIE RÉELLE

```
ordinal : [BASSE, NORMALE, HAUTE, URGENTE]
poids   : [BASSE, NORMALE, HAUTE, URGENTE]
nom     : [BASSE, HAUTE, NORMALE, URGENTE]
ordinal == poids ? true
```

Ici `ordinal` et `poids` coïncident — et c'est justement le cas piège : le jour où quelqu'un insère `CRITIQUE` entre `HAUTE` et `URGENTE` avec un poids de 5, les deux tris divergent silencieusement. Trier sur l'attribut métier rend l'intention explicite et survit aux réordonnancements.

> **Comment choisir entre `comparing` et `comparingInt`**
>
> `Comparator.comparing(Priorite::poids)` avec un `int` passe par l'autoboxage en `Integer` puis par `compareTo` ; `comparingInt(Priorite::poids)` reste sur des primitives. Sur des collections modestes la différence est invisible, mais `comparingInt` est plus juste et évite une allocation par comparaison. À noter : une méthode qui renvoie `Integer` peut renvoyer `null` et faire échouer le tri en pleine exécution — voir la section 3.4.

3.4 Les pièges de `compareTo` (null, type, ordinal figé)

Piège	Ce qui se passe	Solution
`a.compareTo(null)`	`NullPointerException`	`Comparator.nullsFirst(...)` / `nullsLast(...)`, ou tester `null` avant.
Comparer deux enums différents	`ClassCastException`	Erreur de type : ne devrait pas arriver avec des types génériques corrects.
Stocker l'`ordinal()`	Instable : un réordonnancement casse les données	Stocker `name()`, ou un code explicite (chapitre 7.3).
Trier avec un attribut `null`	`NullPointerException` au milieu du tri	`Comparator.nullsLast(comparing(...))`.
Comparer via `toString()`	Faux si `toString()` est redéfini pour l'affichage	Comparer `name()` (section 4.3).

PROGRAMME — REPRODUIRE LES DEUX ERREURS LES PLUS FRÉQUENTES

```
List<Priorite> avecNull = new ArrayList<>();
avecNull.add(Priorite.HAUTE);
avecNull.add(null);
try {
    avecNull.sort(null);
} catch (NullPointerException e) {
    System.out.println("tri avec null  -> NullPointerException");
}
avecNull.sort(Comparator.nullsLast(Comparator.naturalOrder()));
System.out.println("nullsLast      -> " + avecNull);

try {
    System.out.println(((Enum) Priorite.HAUTE).compareTo("chaîne"));
} catch (ClassCastException e) {
    System.out.println("type étranger  -> ClassCastException");
}
```

SORTIE RÉELLE

```
tri avec null  -> NullPointerException
nullsLast      -> [HAUTE, null]
type étranger  -> ClassCastException
```

PROGRAMME — COMPARER UN ATTRIBUT NULLABLE SANS EXPLOSER

```
record Fiche(String nom, Priorite priorite) { }

List<Fiche> fiches = new ArrayList<>(List.of(
        new Fiche("a", Priorite.HAUTE),
        new Fiche("b", null),
        new Fiche("c", Priorite.BASSE)));

fiches.sort(Comparator.comparing(Fiche::priorite,
Comparator.nullsLast(Comparator.naturalOrder())));
System.out.println(fiches);
```

SORTIE RÉELLE

```
[Fiche[nom=c, priorite=BASSE], Fiche[nom=a, priorite=HAUTE], Fiche[nom=b, priorite=null]]
```

> **Pourquoi `null` dans un enum arrive plus souvent qu'on ne croit**
>
> Un champ `Priorite priorite` non renseigné vaut `null` ; un `Map.get()` sur une clé absente renvoie `null` ; un `Optional<Priorite>.orElse(null)` aussi. Dans du code de domaine, la bonne réponse est souvent de **ne pas avoir de null** : donnez une valeur par défaut explicite (`Priorite.NORMALE`) ou utilisez `Optional`. Mais quand vous ne maîtrisez pas la source (base existante, API tierce), `nullsLast` est le filet de sécurité à connaître.

4 Trier des énumérations

> C'est le chapitre que vous êtes venu chercher. Cinq façons de trier, ce que chacune produit réellement, et la réponse à la question du tri « automatique par le nom » — y compris ses deux surprises : les nombres et les accents.

4.1 Les cinq façons de trier, et ce qu'elles donnent

Prenons la même liste de quatre priorités, volontairement mélangée, et appliquons les cinq méthodes usuelles. Toutes les sorties ci-dessous sont réelles.

PROGRAMME — LES CINQ VARIANTES

```
List<Priorite> melange = new ArrayList<>(List.of(
        Priorite.URGENTE, Priorite.BASSE, Priorite.HAUTE, Priorite.NORMALE));

// 1. ordre naturel = ordre de déclaration (= ordinal)
List<Priorite> a = new ArrayList<>(melange);
Collections.sort(a);

// 2. idem, forme moderne
List<Priorite> b = new ArrayList<>(melange);
b.sort(null);

// 3. ordre inverse de l'ordre naturel
List<Priorite> c = new ArrayList<>(melange);
c.sort(Collections.reverseOrder());

// 4. tri par le nom (alphabétique)
List<Priorite> d = new ArrayList<>(melange);
d.sort(Comparator.comparing(Priorite::name));

// 5. tri par un attribut métier
List<Priorite> e = new ArrayList<>(melange);
e.sort(Comparator.comparingInt(Priorite::poids));

System.out.println("avant tri          : " + melange);
System.out.println("1. Collections.sort   : " + a);
System.out.println("2. list.sort(null)    : " + b);
System.out.println("3. ordre inversé      : " + c);
System.out.println("4. par name()         : " + d);
System.out.println("5. par poids          : " + e);
System.out.println("stream().sorted()     : " + melange.stream().sorted().toList());
System.out.println("min / max             : " + Collections.min(melange) + " / " +
Collections.max(melange));
```

SORTIE RÉELLE

```
avant tri          : [URGENTE, BASSE, HAUTE, NORMALE]
1. Collections.sort   : [BASSE, NORMALE, HAUTE, URGENTE]
2. list.sort(null)    : [BASSE, NORMALE, HAUTE, URGENTE]
3. ordre inversé      : [URGENTE, HAUTE, NORMALE, BASSE]
4. par name()         : [BASSE, HAUTE, NORMALE, URGENTE]
5. par poids          : [BASSE, NORMALE, HAUTE, URGENTE]
stream().sorted()     : [BASSE, NORMALE, HAUTE, URGENTE]
min / max             : BASSE / URGENTE
```

Forme	Résultat sur l'exemple	À utiliser quand…
`Collections.sort(liste)`	`[BASSE, NORMALE, HAUTE, URGENTE]`	Code ancien, liste mutable. Fonctionne par héritage de `Comparable`.
`liste.sort(null)`	idem	Forme recommandée depuis Java 8 : `null` signifie « ordre naturel ».
`liste.sort(reverseOrder())`	`[URGENTE, HAUTE, NORMALE, BASSE]`	Ordre décroissant : « les plus urgents d'abord ».
`liste.sort(comparing(E::name))`	`[BASSE, HAUTE, NORMALE, URGENTE]`	Ordre alphabétique indépendant de la déclaration (audit, affichage).
`liste.sort(comparingInt(E::poids))`	`[BASSE, NORMALE, HAUTE, URGENTE]`	Ordre métier explicite, découplé de la déclaration.
`stream().sorted()`	`[BASSE, NORMALE, HAUTE, URGENTE]`	Flux : renvoie une *nouvelle* liste, ne modifie pas l'originale.

`List.of(...)` est immuable : le tri lève une exception

`List.of(Priorite.HAUTE, Priorite.BASSE).sort(null)` échoue avec `UnsupportedOperationException`, car la liste renvoyée par `List.of` ne peut pas être modifiée. Deux solutions : `new ArrayList<>(List.of(...))`, ou passer par un flux avec `stream().sorted().toList()` (qui renvoie elle aussi une liste immuable, mais triée). C'est l'erreur n°1 des débutants sur les tris, tous types confondus.

4.2 Trier par attribut, inverser, combiner

PROGRAMME — LES TROIS GESTES DE BASE SUR UN COMPARATEUR

```
// inverser n'importe quel comparateur
liste.sort(Comparator.comparing(Priorite::name).reversed());

// enchaîner deux critères
liste.sort(Comparator.comparing(Priorite::name).thenComparingInt(Priorite::poids));

// comparer un attribut qui pourrait être null
liste.sort(Comparator.comparing(Fiche::priorite,
        Comparator.nullsLast(Comparator.naturalOrder())));
```

Méthode	Effet
`reversed()`	Inverse le comparateur courant. Attention : appelé au milieu d'une chaîne, il n'inverse que la partie déjà construite. Écrire `comparing(a).thenComparing(b).reversed()` inverse *les deux* critères.
`thenComparing(...)`	Second critère, appliqué seulement quand le premier renvoie 0. Peut recevoir une lambda, une référence de méthode ou un comparateur.
`comparingInt` / `comparingLong` / `comparingDouble`	Évite l'autoboxage pour les attributs numériques. `Comparator.comparing(Priorite::poids)` compile aussi, mais boxe.
`nullsFirst(...)` / `nullsLast(...)`	Place les `null` en tête ou en queue au lieu de lever une exception.
`naturalOrder()` / `reverseOrder()`	L'ordre de l'énumération (déclaration) et son inverse. Disponibles directement en import statique.

Le comparateur ne doit jamais trahir la cohérence

Un comparateur doit être *cohérent avec l'égalité* pour les structures triées : si `compare(a, b) == 0`, alors `a.equals(b)` devrait être vrai. Pour des constantes d'énumération, c'est automatique si vous comparez `name()` ou l'ordre naturel. En revanche, un comparateur qui ne compare qu'un attribut partiel (par exemple `dureeMinutes` seul) rend deux tâches différentes « égales » aux yeux d'un `TreeSet` : la seconde disparaît silencieusement. D'où l'importance d'un critère de départage final, comme dans `ORDRE_METIER` du chapitre 7.1.

4.3 Trier par le nom : `comparing(MonEnum::name)`

C'est la réponse directe à « peut-on trier automatiquement par le nom ? ». Oui, et c'est même simple : `name()` renvoie une `String`, et `String` est `Comparable`. Il n'existe aucune méthode dédiée dans l'API — c'est un comparateur comme un autre.

PROGRAMME

```
List<Priorite> melange = new ArrayList<>(List.of(
        Priorite.URGENTE, Priorite.BASSE, Priorite.HAUTE, Priorite.NORMALE));

List<Priorite> parNom = new ArrayList<>(melange);
parNom.sort(Comparator.comparing(Priorite::name));

List<Priorite> parNomDesc = new ArrayList<>(melange);
parNomDesc.sort(Comparator.comparing(Priorite::name).reversed());

System.out.println("par name()         : " + parNom);
System.out.println("par name() inversé : " + parNomDesc);
System.out.println("valeurs triées     : " + Arrays.stream(Priorite.values())
        .sorted(Comparator.comparing(Enum::name)).toList());
```

SORTIE RÉELLE

```
par name()         : [BASSE, HAUTE, NORMALE, URGENTE]
par name() inversé : [URGENTE, NORMALE, HAUTE, BASSE]
valeurs triées     : [BASSE, HAUTE, NORMALE, URGENTE]
```

Trois écritures équivalentes

`Comparator.comparing(Priorite::name)`, `Comparator.comparing(Enum::name)` et `Comparator.comparing(p -> p.name())` produisent le même résultat. La première est la plus explicite ; la deuxième fonctionne pour n'importe quel enum ; la troisième est utile quand on veut enchaîner avec un second critère typé. Évitez `Comparator.comparing(Object::toString)` : si `toString()` est redéfini pour l'affichage (chapitre 1.3), vous ne triez plus par nom.

Le tri par nom n'est pas « automatique » au sens où on l'entend souvent

Aucune structure ne trie par nom sans qu'on le demande : ni `Collections.sort`, ni `TreeSet`, ni `EnumSet`, ni `stream().sorted()`. Tous utilisent l'ordre **naturel**, c'est-à-dire la déclaration. Il n'existe pas en Java d'annotation ou d'option « trie par nom ». Il faut écrire le comparateur — ce qui a l'avantage d'être visible dans le code, donc vérifiable en revue.

4.4 Le tri « naturel » : nombres, accents, casse

L'ordre alphabétique de `String.compareTo` est un ordre **lexicographique sur les unités de code UTF-16**. Il ne correspond pas à l'idée intuitive qu'on se fait d'un tri alphabétique. Trois démonstrations, toutes réelles.

Les nombres sont comparés caractère par caractère

PROGRAMME

```
// enum Etape { ETAPE_1, ETAPE_2, ETAPE_10, ETAPE_11, ETAPE_3 }
List<Etape> l = new ArrayList<>(List.of(Etape.values()));
System.out.println("déclaration : " + l);
l.sort(Comparator.comparing(Enum::name));
System.out.println("par name()  : " + l);
```

SORTIE RÉELLE

```
déclaration : [ETAPE_1, ETAPE_2, ETAPE_10, ETAPE_11, ETAPE_3]
par name()  : [ETAPE_1, ETAPE_10, ETAPE_11, ETAPE_2, ETAPE_3]
```

`ETAPE_10` est placé avant `ETAPE_2` parce que le caractère `'1'` précède le caractère `'2'` — et que la comparaison s'arrête là. Ce n'est pas un bug : c'est le comportement défini de `String.compareTo`. Deux solutions.

SOLUTION 1 — REMBOURRER LES NUMÉROS DANS LE NOM DE LA CONSTANTE

```
enum Etape {
    ETAPE_01, ETAPE_02, ETAPE_03, ETAPE_10, ETAPE_11    // zéros non significatifs
}

System.out.println(List.of("ETAPE_01", "ETAPE_02", "ETAPE_03", "ETAPE_10", "ETAPE_11")
        .stream().sorted().toList());
```

SORTIE RÉELLE

```
[ETAPE_01, ETAPE_02, ETAPE_03, ETAPE_10, ETAPE_11]
```

SOLUTION 2 — UN COMPARATEUR « NATUREL » QUI COMPARE LES BLOCS DE CHIFFRES COMME DES NOMBRES

```
Comparator<String> naturel = (x, y) -> {
    int i = 0, j = 0;
    while (i < x.length() && j < y.length()) {
        char cx = x.charAt(i), cy = y.charAt(j);
        if (Character.isDigit(cx) && Character.isDigit(cy)) {
            int si = i, sj = j;
            while (i < x.length() && Character.isDigit(x.charAt(i))) i++;
            while (j < y.length() && Character.isDigit(y.charAt(j))) j++;
            int nx = Integer.parseInt(x.substring(si, i));
            int ny = Integer.parseInt(y.substring(sj, j));
            if (nx != ny) return Integer.compare(nx, ny);
        } else {
            if (cx != cy) return Character.compare(cx, cy);
            i++; j++;
        }
    }
    return Integer.compare(x.length() - i, y.length() - j);
};

l.sort(Comparator.comparing(Enum::name, naturel));
```

SORTIE RÉELLE

```
par nom naturel : [ETAPE_1, ETAPE_2, ETAPE_3, ETAPE_10, ETAPE_11]
```

Y a-t-il un « tri naturel » tout fait dans le JDK ?

Non. La classe `java.text.Collator` sait faire un tri linguistique (accents, casse, contractions), mais elle compare des mots, pas des blocs numériques : `Collator` classe `ETAPE_10` avant `ETAPE_2`. Le comparateur « alphanumérique » du type *natural sort* n'existe pas dans la bibliothèque standard ; il faut l'écrire (ci-dessus, une trentaine de lignes) ou rembourrer les numéros. Pour des noms de constantes, la solution la plus robuste reste le rembourrage : elle rend l'ordre alphabétique identique à l'ordre logique.

Les accents se classent après le Z

PROGRAMME

```
// enum Accent { ETE, ELEVE, ÉTÉ, EAU, ZEBRE }
List<Accent> la = new ArrayList<>(List.of(Accent.values()));
la.sort(Comparator.comparing(Enum::name));
System.out.println("compareTo (UTF-16) : " + la);

Collator fr = Collator.getInstance(Locale.FRENCH);
fr.setStrength(Collator.PRIMARY);
List<Accent> lb = new ArrayList<>(List.of(Accent.values()));
lb.sort(Comparator.comparing(Enum::name, fr));
System.out.println("Collator FRENCH    : " + lb);
```

SORTIE RÉELLE

```
compareTo (UTF-16) : [EAU, ELEVE, ETE, ZEBRE, ÉTÉ]
Collator FRENCH    : [EAU, ELEVE, ETE, ÉTÉ, ZEBRE]
```

En UTF-16, `É` (U+00C9) a un code supérieur à celui de `Z` (U+005A) : `ÉTÉ` se retrouve donc à la fin. C'est mathématiquement correct et humainement faux. `Collator` corrige cela : `ÉTÉ` se place après `ETE`, et `ZEBRE` reste en dernier.

Casse : l'ordre ASCII place les majuscules avant les minuscules

Sur des noms de constantes, tout est en majuscules : le problème ne se pose pas. Mais dès que vous triez des `toString()` ou des libellés, sachez que `String.compareTo` classe `Beta` avant `alpha` (car `'B'` = 66 < `'a'` = 97). Le correctif est `String.CASE_INSENSITIVE_ORDER`, ou `Collator` avec `setStrength(Collator.SECONDARY)`. Le programme de vérification joint à cette leçon (`demo/`) affiche les deux :

RÉSULTAT

```
compareTo             : [Beta, Delta, alpha, gamma]
CASE_INSENSITIVE_ORDER : [alpha, Beta, Delta, gamma]
```

4.5 Structures déjà triées : `EnumSet`, `EnumMap`, `TreeSet`, `TreeMap`

Une énumération a un avantage décisif sur une collection d'objets : elle est finie, connue à la compilation, et son ordre est stable. Le JDK fournit pour cela deux structures spécialisées, qui sont à la fois plus rapides et **toujours ordonnées**.

Structure	Ordre d'itération	Ce qu'il faut savoir
`EnumSet`	Ordre de déclaration, toujours	Représenté par un ou des `long` (masque de bits) : ajout et test en temps constant, sans hachage. Implémentations : `RegularEnumSet` (≤ 64 constantes) et `JumboEnumSet` (au-delà).
`EnumMap`	Ordre de déclaration des clés	Tableau indexé par `ordinal()` : pas de hachage, pas de collision. Léger et rapide. Toutes les clés doivent être du même enum.
`TreeSet`	Ordre naturel, ou ordre du comparateur fourni	Utile pour un ordre différent (inversé, par nom). Plus coûteux : arbre rouge-noir.
`TreeMap`	Idem, sur les clés	Donne accès à `headMap`, `tailMap`, `subMap` : « toutes les priorités jusqu'à NORMALE ».
`HashSet` / `HashMap`	**Aucun ordre garanti**	À éviter quand l'ordre compte : l'affichage peut changer d'une exécution à l'autre.

PROGRAMME — LES QUATRE STRUCTURES SUR LES MÊMES DONNÉES

```
List<Priorite> melange = List.of(Priorite.URGENTE, Priorite.BASSE, Priorite.HAUTE);

EnumSet<Priorite> ensemble = EnumSet.copyOf(melange);
System.out.println("EnumSet.copyOf      : " + ensemble);
System.out.println("EnumSet.allOf       : " + EnumSet.allOf(Priorite.class));
System.out.println("EnumSet.range       : " + EnumSet.range(Priorite.NORMALE, Priorite.URGENTE));
System.out.println("EnumSet.complementOf: " + EnumSet.complementOf(EnumSet.of(Priorite.HAUTE)));

EnumMap<Priorite, String> parPriorite = new EnumMap<>(Priorite.class);
parPriorite.put(Priorite.HAUTE, "orange");
parPriorite.put(Priorite.BASSE, "vert");
parPriorite.put(Priorite.URGENTE, "rouge");
System.out.println("EnumMap             : " + parPriorite);
System.out.println("EnumMap.keySet()    : " + parPriorite.keySet());

TreeSet<Priorite> arbre = new TreeSet<>(melange);
System.out.println("TreeSet             : " + arbre
        + " (first=" + arbre.first() + ", last=" + arbre.last() + ")");
TreeSet<Priorite> arbreInverse = new TreeSet<>(Collections.reverseOrder());
arbreInverse.addAll(melange);
System.out.println("TreeSet inversé     : " + arbreInverse);

TreeMap<Priorite, String> carte = new TreeMap<>(parPriorite);
System.out.println("TreeMap             : " + carte);
System.out.println("headMap(NORMALE)    : " + carte.headMap(Priorite.NORMALE));
```

SORTIE RÉELLE

```
EnumSet.copyOf      : [BASSE, HAUTE, URGENTE]
EnumSet.allOf       : [BASSE, NORMALE, HAUTE, URGENTE]
EnumSet.range       : [NORMALE, HAUTE, URGENTE]
EnumSet.complementOf: [BASSE, NORMALE, URGENTE]
EnumMap             : {BASSE=vert, HAUTE=orange, URGENTE=rouge}
EnumMap.keySet()    : [BASSE, HAUTE, URGENTE]
TreeSet             : [BASSE, HAUTE, URGENTE] (first=BASSE, last=URGENTE)
TreeSet inversé     : [URGENTE, HAUTE, BASSE]
TreeMap             : {BASSE=vert, HAUTE=orange, URGENTE=rouge}
headMap(NORMALE)    : {BASSE=vert}
```

> **Pourquoi `EnumSet` et `EnumMap` plutôt qu'un `HashSet`**
>
> Trois raisons, dans l'ordre d'importance. **1. L'ordre** : l'itération suit l'ordre de déclaration, donc elle est déterministe et testable — un `HashSet` peut changer d'ordre d'une exécution à l'autre. **2. La mémoire** : un masque de bits (un ou plusieurs `long`) au lieu d'une table de hachage. **3. La vitesse**, mais avec des nuances : sur notre machine, deux millions d'ajouts et de tests prennent ≈ 7,4 ms avec `EnumSet` contre ≈ 10,8 ms avec `HashSet` ; en revanche `EnumMap` et `HashMap` sont dans le même ordre de grandeur (3-4 ms), la JIT optimisant très bien les deux. Corollaire pratique : `EnumSet` est un gain net, `EnumMap` est surtout un gain de *lisibilité et de déterminisme*. Dans les deux cas, n'écrivez pas `HashMap<MonEnum, …>`.

4.6 Trier des objets qui contiennent une énumération

Le cas réel : vous ne triez pas des constantes, vous triez des *objets* dont un champ est une énumération. Deux méthodes, et il faut choisir consciemment.

FICHIER — `TACHE.JAVA`, AVEC UN ORDRE MÉTIER DÉCLARÉ UNE SEULE FOIS

```
public record Tache(String nom, Priorite priorite, int dureeMinutes) implements Comparable<Tache>
{

    public static final Comparator<Tache> ORDRE_METIER =
            Comparator.comparing(Tache::priorite).reversed()   // les plus urgentes d'abord
                      .thenComparingInt(Tache::dureeMinutes)   // puis les plus courtes
                      .thenComparing(Tache::nom);              // puis l'ordre alphabétique

    @Override
    public int compareTo(Tache autre) {
        return ORDRE_METIER.compare(this, autre);
    }

    @Override
    public String toString() {
        return nom + " (" + priorite.name() + ", " + dureeMinutes + " min)";
    }
}
```

PROGRAMME

```
List<Tache> taches = new ArrayList<>(List.of(
        new Tache("Rapport", Priorite.BASSE, 120),
        new Tache("Incident", Priorite.URGENTE, 30),
        new Tache("Revue de code", Priorite.HAUTE, 45),
        new Tache("Sauvegarde", Priorite.NORMALE, 15),
        new Tache("Audit", Priorite.HAUTE, 45)));

System.out.println("ordre d'entrée     : " + taches);
System.out.println("Collections.sort   : " + taches.stream().sorted().toList());
System.out.println("par priorité seule : " + taches.stream()
        .sorted(Comparator.comparing(Tache::priorite)).toList());
```

SORTIE RÉELLE

```
ordre d'entrée     : [Rapport (BASSE, 120 min), Incident (URGENTE, 30 min), Revue de code (HAUTE, 45
min), Sauvegarde (NORMALE, 15 min), Audit (HAUTE, 45 min)]
Collections.sort   : [Incident (URGENTE, 30 min), Audit (HAUTE, 45 min), Revue de code (HAUTE, 45
min), Sauvegarde (NORMALE, 15 min), Rapport (BASSE, 120 min)]
par priorité seule : [Rapport (BASSE, 120 min), Sauvegarde (NORMALE, 15 min), Revue de code (HAUTE, 45
min), Audit (HAUTE, 45 min), Incident (URGENTE, 30 min)]
```

Remarquez les deux premiers résultats : `Audit` passe avant `Revue de code` alors que les deux sont en `HAUTE`, parce que le second critère (durée) ne les départage pas (45 = 45) et que le troisième (nom)

tranche : `"Audit"` < `"Revue de code"`. C'est ce qu'on appelle un **ordre total** : aucun couple d'éléments n'est ambigu. Sans le troisième critère, l'ordre entre deux tâches identiques en priorité et en durée dépendrait de l'algorithme de tri — un résultat non reproductible, donc impossible à tester.

Le comparateur, écrit une seule fois

La bonne pratique qui ressort de cet exemple : un ordre métier se déclare **une fois**, dans une constante `static final` du type concerné, puis se réutilise partout. Vous évitez ainsi les trois copies divergentes du même tri (l'écran, l'export, le rapport) qui finissent par ne plus classer de la même façon. Et comme `Comparator` expose `thenComparing` et `reversed`, vous composez ces ordres au lieu de les réécrire.

PROGRAMME — DÉCLARER LES TROIS ORDRES UNE FOIS, PUIS LES COMPOSER

```
List<Tache> taches = List.of(
        new Tache("Rapport", Priorite.BASSE, 120),
        new Tache("Incident", Priorite.URGENTE, 30),
        new Tache("Audit", Priorite.HAUTE, 45),
        new Tache("Migration", Priorite.URGENTE, 240),
        new Tache("Relance", Priorite.BASSE, 10));

// les trois ordres, réutilisables
Comparator<Tache> parUrgence = Comparator.comparing(Tache::priorite).reversed();
Comparator<Tache> parDuree   = Comparator.comparingInt(Tache::dureeMinutes);
Comparator<Tache> parNom     = Comparator.comparing(Tache::nom);

taches.stream().sorted(parUrgence).forEach(t -> System.out.println("urgence : " + t));
taches.stream().sorted(parDuree).forEach(t -> System.out.println("durée   : " + t));
taches.stream().sorted(parNom).forEach(t -> System.out.println("nom     : " + t));
taches.stream().sorted(parUrgence.thenComparing(parDuree).thenComparing(parNom))
      .forEach(t -> System.out.println("métier  : " + t));
```

SORTIE RÉELLE

```
urgence : Incident (URGENTE, 30 min)
urgence : Migration (URGENTE, 240 min)
urgence : Audit (HAUTE, 45 min)
urgence : Rapport (BASSE, 120 min)
urgence : Relance (BASSE, 10 min)
durée   : Relance (BASSE, 10 min)
durée   : Incident (URGENTE, 30 min)
durée   : Audit (HAUTE, 45 min)
durée   : Rapport (BASSE, 120 min)
durée   : Migration (URGENTE, 240 min)
nom     : Audit (HAUTE, 45 min)
nom     : Incident (URGENTE, 30 min)
nom     : Migration (URGENTE, 240 min)
nom     : Rapport (BASSE, 120 min)
nom     : Relance (BASSE, 10 min)
métier  : Incident (URGENTE, 30 min)
métier  : Migration (URGENTE, 240 min)
métier  : Audit (HAUTE, 45 min)
métier  : Relance (BASSE, 10 min)
métier  : Rapport (BASSE, 120 min)
```

> **`Comparable` ou `Comparator` ?**
>
> **`Comparable`** quand il existe *un* ordre naturel évident pour le type (une priorité se compare à une priorité) : `compareTo` est alors rigide et disponible partout. **`Comparator`** quand plusieurs ordres coexistent (par nom, par date, par priorité) : autant de constantes statiques que d'ordres, sans toucher au type. Le code de la section 4.2 combine les deux : une constante `ORDRE_METIER` réutilisable, et un `compareTo` qui l'expose comme ordre naturel.

Cinq éléments, pas trois : le jeu de test qui distingue vraiment deux tris

Avec seulement trois tâches, les quatre tris ci-dessus se ressembleraient au point de laisser croire qu'ils sont équivalents. Il faut un cas où *chaque* critère change le résultat : ici « Migration » (urgente mais longue) et « Relance » (secondaire mais courte) suffisent à faire diverger les ordres par urgence, par durée et l'ordre métier. Règle pratique : pour valider un comparateur, écrivez un test où deux éléments sont à égalité sur le premier critère et différents sur le second. C'est le seul qui prouve que le départage fonctionne.

5 Construire une hiérarchie

> « Peut-on créer une énumération avec une hiérarchie, automatique par le nom ? » La réponse honnête tient en trois parties : **non**, Java n'a aucune hiérarchie implicite entre constantes ; **oui**, on peut en construire une explicitement, et c'est très propre ; et **oui**, l'ordre de déclaration fournit une hiérarchie linéaire gratuite — à condition de nommer ses constantes de façon cohérente.

5.1 Pourquoi un enum ne peut pas hériter

Un `enum` hérite *déjà* de `java.lang.Enum` : la place est prise. Java n'a pas d'héritage multiple de classes, donc aucune énumération ne peut étendre une autre classe, ni une autre énumération. Les trois tentatives échouent à la compilation :

Tentative	Ce que dit `javac`
`enum E extends Base`	`error: '{' expected` puis `error: enum constant expected here`
`enum Enfant extends ParentEnum`	Même refus : `Enum` est déjà la superclasse, et elle est `final`.
`enum E implements Comparable<E>` avec son propre `compareTo`	`error: compareTo(E) cannot override compareTo(E) in Enum` — *overridden method is final*

> **Ce qui reste possible, et qui suffit presque toujours**
>
> Un enum peut **implémenter des interfaces** (chapitre 2.3), **porter des champs** de son propre type (c'est la clé de la hiérarchie de la section 5.2), et **déclarer des méthodes abstraites** implémentées par chaque constante (chapitre 2.2). Ces trois mécanismes couvrent l'immense majorité des besoins qu'on croit devoir résoudre par l'héritage.

5.2 La hiérarchie explicite : un champ `parent`

L'idée est simple : chaque constante porte une référence vers son parent, `null` pour une racine. Comme une constante ne peut référencer qu'une constante *déjà déclarée* (chapitre 2.5), il suffit de déclarer les parents avant les enfants — c'est-à-dire de haut en bas.

FICHIER — `CATEGORIE.JAVA` : UN ARBRE À TROIS NIVEAUX

```
import java.util.ArrayList;
import java.util.List;

public enum Categorie {
    INCIDENT(null),
    RESEAU(INCIDENT),
    SERVEUR(INCIDENT),
    PARE_FEU(RESEAU),
    DNS(RESEAU),
    DISQUE(SERVEUR),
    DEMANDE(null),
    MATERIEL(DEMANDE);

    private final Categorie parent;

    Categorie(Categorie parent) {
        this.parent = parent;
    }

    public Categorie parent() {
        return parent;
    }

    public boolean estRacine() {
        return parent == null;
    }

    /** Profondeur dans l'arbre : 0 pour une racine, 1 pour un enfant, etc. */
    public int profondeur() {
        int d = 0;
        for (Categorie c = parent; c != null; c = c.parent) {
            d++;
        }
        return d;
    }

    /** Chaîne des ancêtres, de la racine jusqu'à ce nœud. */
    public List<Categorie> chemin() {
        List<Categorie> chemin = new ArrayList<>();
        for (Categorie c = this; c != null; c = c.parent) {
            chemin.add(0, c);
        }
        return chemin;
    }

    /** Vrai si cette catégorie est {@code ancetre} ou en descend. */
    public boolean sous(Categorie ancetre) {
        for (Categorie c = this; c != null; c = c.parent) {
            if (c == ancetre) {
                return true;
            }
        }
        return false;
    }
}
```

PROGRAMME

```
for (Categorie c : Categorie.values()) {
    System.out.printf("%-10s parent=%-10s profondeur=%d chemin=%s%n",
            c, c.parent(), c.profondeur(), c.chemin());
}
System.out.println("DNS sous INCIDENT : " + Categorie.DNS.sous(Categorie.INCIDENT));
System.out.println("DNS sous SERVEUR  : " + Categorie.DNS.sous(Categorie.SERVEUR));
```

SORTIE RÉELLE

```
INCIDENT   parent=null       profondeur=0 chemin=[INCIDENT]
RESEAU     parent=INCIDENT   profondeur=1 chemin=[INCIDENT, RESEAU]
SERVEUR    parent=INCIDENT   profondeur=1 chemin=[INCIDENT, SERVEUR]
PARE_FEU   parent=RESEAU     profondeur=2 chemin=[INCIDENT, RESEAU, PARE_FEU]
DNS        parent=RESEAU     profondeur=2 chemin=[INCIDENT, RESEAU, DNS]
DISQUE     parent=SERVEUR    profondeur=2 chemin=[INCIDENT, SERVEUR, DISQUE]
DEMANDE    parent=null       profondeur=0 chemin=[DEMANDE]
MATERIEL   parent=DEMANDE    profondeur=1 chemin=[DEMANDE, MATERIEL]
DNS sous INCIDENT : true
DNS sous SERVEUR  : false
```

Pourquoi ce modèle est meilleur qu'un héritage

Avec un héritage classique, « DNS est un RESEAU » serait figé dans le type : impossible de déplacer `DNS` sous `SERVEUR` sans recompiler et sans casser les `instanceof`. Ici, la hiérarchie est une **donnée** : on peut la parcourir, l'afficher, la valider, la sérialiser, l'alimenter depuis un fichier de configuration. Et elle reste vérifiée à la compilation, puisqu'on ne peut référencer qu'une constante existante.

Trois pièges de la hiérarchie par champ

1. Les cycles. Rien n'empêche théoriquement d'écrire `A(B), B(A)` — sauf que le compilateur refuse la référence avant déclaration (`illegal forward reference`). Le cycle est donc impossible *par construction*, ce qui est une bonne nouvelle : vos boucles `for (c = parent; …)` ne peuvent pas boucler à l'infini.

2. La suppression d'un nœud. Si vous supprimez `RESEAU`, ses enfants ne compilent plus. C'est voulu : vous êtes forcé de traiter le problème. Une migration de données est alors nécessaire pour les lignes qui pointaient vers l'ancienne valeur.

3. Le parcours récursif. Les méthodes ci-dessus sont itératives et bornées par la profondeur. Si vous ajoutez un parcours d'enfants (section 5.3), attention à ne pas reconstruire l'arbre à chaque appel dans une boucle chaude : calculez-le une fois.

5.3 Parcourir : profondeur, racines, sous-arbres, enfants

PROGRAMME — LES QUATRE VUES UTILES D'UN ARBRE

```
// 1. enfants, groupeés par parent
EnumMap<Categorie, List<Categorie>> enfants = new EnumMap<>(Categorie.class);
for (Categorie c : Categorie.values()) {
    if (c.parent() != null) {
        enfants.computeIfAbsent(c.parent(), k -> new ArrayList<>()).add(c);
    }
}
System.out.println("enfants : " + enfants);

// 2. racines
System.out.println("racines : " +
Arrays.stream(Categorie.values()).filter(Categorie::estRacine).toList());

// 3. sous-arbre complet d'un nœud
System.out.println("sous-arbre INCIDENT : " + Arrays.stream(Categorie.values())
        .filter(c -> c.sous(Categorie.INCIDENT)).toList());

// 4. tri par niveau, puis alphabétique à niveau égal
List<Categorie> parcours = new ArrayList<>(List.of(Categorie.values()));
parcours.sort(Comparator.comparingInt(Categorie::profondeur).thenComparing(Categorie::name));
System.out.println("tri profondeur+nom : " + parcours);
```

SORTIE RÉELLE

```
enfants : {INCIDENT=[RESEAU, SERVEUR], RESEAU=[PARE_FEU, DNS], SERVEUR=[DISQUE], DEMANDE=[MATERIEL]}
racines : [INCIDENT, DEMANDE]
sous-arbre INCIDENT : [INCIDENT, RESEAU, SERVEUR, PARE_FEU, DNS, DISQUE]
tri profondeur+nom : [DEMANDE, INCIDENT, MATERIEL, RESEAU, SERVEUR, DISQUE, DNS, PARE_FEU]
```

Le tri par profondeur illustre un point important : **l'ordre de déclaration et l'ordre de l'arbre ne coïncident pas**. `Categorie.values()` suit la déclaration (`INCIDENT, RESEAU, SERVEUR, …`), qui est un parcours « en largeur » improvisé. Pour afficher un arbre, il faut trier par profondeur — et pour un vrai affichage indenté, il faut un parcours en profondeur.

PROGRAMME — AFFICHAGE INDENTÉ (PARCOURS EN PROFONDEUR)

```
static void afficher(Categorie racine, EnumMap<Categorie, List<Categorie>> enfants, String
prefixe) {
    System.out.println(prefixe + racine);
    for (Categorie enfant : enfants.getOrDefault(racine, List.of())) {
        afficher(enfant, enfants, prefixe + "    ");
    }
}

for (Categorie racine : Categorie.values()) {
    if (racine.estRacine()) {
        afficher(racine, enfants, "");
    }
}
```

SORTIE RÉELLE

```
INCIDENT
    RESEAU
        PARE_FEU
        DNS
    SERVEUR
        DISQUE
DEMANDE
    MATERIEL
```

5.4 Quand `sealed` + `record` remplacent l'enum

Il existe une alternative à la hiérarchie par champ : depuis Java 17, une interface ou une classe `sealed` peut déclarer la liste exacte de ses sous-types. Combinée aux `record`, elle donne une hiérarchie *vraie* — avec des types différents, donc des champs différents — tout en conservant l'exhaustivité du `switch`.

FICHIER — HIÉRARCHIE SCELLÉE

```
public sealed interface Evenement permits Creation, Modification, Suppression { }

public record Creation(String nom) implements Evenement { }
public record Modification(String nom, String ancienNom) implements Evenement { }
public record Suppression(String nom, String motif) implements Evenement { }
```

PROGRAMME — SWITCH EXHAUSTIF SUR UNE HIÉRARCHIE SCELLÉE

```
static String decrire(Evenement e) {
    return switch (e) {
        case Creation c        -> "création de " + c.nom();
        case Modification m    -> "renommage " + m.ancienNom() + " -> " + m.nom();
        case Suppression s     -> "suppression de " + s.nom() + " (" + s.motif() + ")";
    };
}
```

Besoin	Énumération (+ champ `parent`)	Hiérarchie `sealed` + `record`
Constantes de même forme	**Idéal** : une seule déclaration, un constructeur commun.	Verbeux : une classe par cas, même quand les champs sont identiques.
Données différentes par cas	Mal adapté : il faudrait des champs inutilisés et des `null`.	**Idéal** : chaque `record` déclare exactement ses champs.
`values()`, `valueOf`, `EnumSet`, `EnumMap`	**Offerts** par le langage.	Absents : il faut tenir une liste à la main.
Persistance par nom	**Trivial** (`name()`).	Nécessite un mécanisme maison (type + champs).
Exhaustivité vérifiée	Oui, sur `switch` d'énumération.	Oui, sur `switch` de hiérarchie scellée.
Hiérarchie à plusieurs niveaux	Par champ `parent` (données).	Par sous-interfaces scellées (types).

> **La règle de choix, en une phrase**
>
> Si vos cas ne diffèrent que par des **valeurs**, c'est une énumération. S'ils diffèrent par leur **structure** (certains ont trois champs, d'autres aucun), c'est une hiérarchie scellée. Et si vous avez besoin d'`EnumSet`, de `EnumMap` ou d'un stockage en base par nom, l'énumération gagne presque toujours.

5.5 La hiérarchie « automatique » par le nom : ce qui existe vraiment

Venons-en à la question posée telle quelle : existe-t-il un mécanisme qui déduit automatiquement une hiérarchie à partir du *nom* des constantes ? Voici l'état exact de l'art en Java.

Ce qu'on pourrait espérer	Existe ?	Réalité
Un enum enfant d'un autre enum	Non	Interdit par le langage : un enum hérite déjà de `Enum`.
Un ordre hiérarchique déduit des noms (`RESEAU_DNS` sous `RESEAU`)	Non	Aucune API du JDK ne fait cela. Le préfixe `RESEAU_` n'est qu'une convention de lecture : `valueOf` et `ordinal` l'ignorent totalement.
Un tri « par nom » automatique	Oui, mais explicite	`Comparator.comparing(E::name)` — chapitre 4.3. Ce n'est pas automatique au sens d'implicite.
Un tri alphanumérique intelligent	Non	À écrire soi-même (chapitre 4.4), ou à obtenir en rembourrant les numéros.
Un tri linguistique (accents, casse)	Oui	`java.text.Collator` — chapitre 4.4.
Une hiérarchie construite à partir du nom, automatiquement	Possible, mais à coder	On peut analyser les préfixes (`RESEAU_DNS` → parent `RESEAU`) dans une méthode statique. C'est une convention maison, pas une fonctionnalité du langage — et c'est fragile dès qu'un nom contient un souligné « métier ».

La seule « hiérarchie automatique » que Java offre : l'ordre linéaire

L'ordre de déclaration *est* une hiérarchie, au sens où il classe les constantes de façon totale et automatique, sans que vous ayez à écrire une ligne. C'est ce que fait `compareTo`. La condition pour que cette hiérarchie soit *correcte* est une convention de nommage et de déclaration cohérente :

FICHIER — DÉCLARER DU PLUS PETIT AU PLUS GRAND, ET LE DOCUMENTER

```
/**
 * Niveau de gravité d'un journal.
 * L'ordre naturel (compareTo) va du moins grave au plus grave :
 * DEBUG < INFO < AVERTISSEMENT < ERREUR < FATAL.
 */
public enum Gravite {
    DEBUG,
    INFO,
    AVERTISSEMENT,
    ERREUR,
    FATAL;

    /** Vrai si ce niveau est au moins aussi grave que {@code autre}. */
    public boolean auMoins(Gravite autre) {
        return this.compareTo(autre) >= 0;
    }

    /** Filtre : garde-t-on un message émis à ce niveau ? */
    public boolean passe(Gravite seuil) {
        return this.compareTo(seuil) >= 0;
    }
}
```

PROGRAMME

```
Gravite seuil = Gravite.AVERTISSEMENT;
for (Gravite g : Gravite.values()) {
    System.out.println(g + " affiché par un seuil AVERTISSEMENT ? " + g.passe(seuil));
}
System.out.println("tri naturel : " + Arrays.stream(Gravite.values()).sorted().toList());
System.out.println("tri par nom  : " + Arrays.stream(Gravite.values())
        .sorted(Comparator.comparing(Enum::name)).toList());
System.out.println("est-ce pareil ? " + Arrays.equals(
        Arrays.stream(Gravite.values()).sorted().toArray(),
        Arrays.stream(Gravite.values()).sorted(Comparator.comparing(Enum::name)).toArray()));
```

SORTIE RÉELLE

```
DEBUG affiché par un seuil AVERTISSEMENT ? false
INFO affiché par un seuil AVERTISSEMENT ? false
AVERTISSEMENT affiché par un seuil AVERTISSEMENT ? true
ERREUR affiché par un seuil AVERTISSEMENT ? true
FATAL affiché par un seuil AVERTISSEMENT ? true
tri naturel : [DEBUG, INFO, AVERTISSEMENT, ERREUR, FATAL]
tri par nom  : [AVERTISSEMENT, DEBUG, ERREUR, FATAL, INFO]
est-ce pareil ? false
```

La démonstration qui tue : ordre naturel ≠ ordre alphabétique

Sur cet exemple — qui ressemble trait pour trait à l'énumération `java.util.logging.Level` de la bibliothèque standard — les deux ordres sont **radicalement différents** : `AVERTISSEMENT` passe en premier alphabétiquement, alors qu'il est troisième en gravité. Si vous triez par nom une liste de niveaux de gravité, vous affichez les erreurs après les avertissements et les `FATAL` au milieu. C'est exactement l'erreur que le chapitre 4.3 met en garde de commettre « pour faire joli ».

La hiérarchie par préfixe : possible, mais à assumer

Si vous voulez vraiment que le nom *porte* la hiérarchie, c'est faisable — en clair, avec ses limites. Voici une implémentation complète, courte, et sans magie :

FICHIER — HIÉRARCHIE DÉDUITE DU NOM PAR CONVENTION `PARENT_ENFANT`

```
import java.util.Optional;

public enum Code {
    INCIDENT,
    INCIDENT_RESEAU,
    INCIDENT_RESEAU_DNS,
    INCIDENT_SERVEUR,
    DEMANDE;

    /** Parent déduit du nom : on retire le dernier segment après le souligné. */
    public Optional<Code> parent() {
        String nom = name();
        int i = nom.lastIndexOf('_');
        if (i < 0) {
            return Optional.empty();
        }
        return Optional.of(valueOf(nom.substring(0, i)));
    }

    public int profondeur() {
        return name().split("_").length - 1;
    }
}
```

PROGRAMME

```
for (Code c : Code.values()) {
    System.out.printf("%-22s parent=%-16s profondeur=%d%n",
            c, c.parent().orElse(null), c.profondeur());
}
System.out.println("par name() : " + Arrays.stream(Code.values())
        .sorted(Comparator.comparing(Enum::name)).toList());
```

SORTIE RÉELLE

```
INCIDENT               parent=null             profondeur=0
INCIDENT_RESEAU        parent=INCIDENT         profondeur=1
INCIDENT_RESEAU_DNS    parent=INCIDENT_RESEAU  profondeur=2
INCIDENT_SERVEUR       parent=INCIDENT         profondeur=1
DEMANDE                parent=null             profondeur=0
par name() : [DEMANDE, INCIDENT, INCIDENT_RESEAU, INCIDENT_RESEAU_DNS, INCIDENT_SERVEUR]
```

Pourquoi cette approche est séduisante et pourquoi elle casse

Séduisante : rien à maintenir, la hiérarchie est visible dans le nom, et le tri par nom donne un parcours d'arbre correct (les parents avant les enfants, comme le montre la dernière ligne). **Fragile** pour quatre raisons, toutes rencontrées en projet :

1. Un souligné « métier » casse tout : `INCIDENT_HORS_SERVICE` devient un enfant de `INCIDENT_HORS`, qui n'existe pas → `IllegalArgumentException` au premier appel de `parent()`.
2. Renommer une constante (ou son parent) change la hiérarchie, sans que rien ne le signale à la compilation.
3. Deux hiérarchies dans le même enum deviennent impossibles (le nom ne peut encoder qu'un axe).
4. L'ordre alphabétique ne respecte pas l'ordre de déclaration : ici c'est sans gravité, mais si `ordinal` sert quelque part, les deux ordres divergent.

Conclusion : préférez le champ `parent` explicite (section 5.2). Il coûte une ligne par constante, il est vérifié à la compilation, et il survit aux renommages.

Variante robuste : la hiérarchie par index d'arbre

Quand la hiérarchie est *parfaitement régulière* — un arbre où chaque niveau a une position connue, comme un plan comptable ou un découpage géographique — on peut la représenter par un **tableau d'indices** plutôt que par un pointeur. Chaque constante porte `{1}`, `{1, 2}`, `{1, 2, 3}`… : le parent est la constante dont le chemin est ce tableau privé de son dernier élément.

EXTRAIT DE L'IMPLÉMENTATION

```
public enum Noeud {
    RACINE(1),
    BRANCHE(1, 1),
    FEUILLE(1, 1, 1);

    private final int[] chemin;

    Noeud(int... chemin) { this.chemin = chemin; }

    public boolean estEnfantDe(Noeud autre) {
        if (chemin.length != autre.chemin.length + 1) return false;
        for (int i = 0; i < autre.chemin.length; i++)
            if (chemin[i] != autre.chemin[i]) return false;
        return true;
    }
}
```

Avantages : l'ordre de déclaration suffit à garantir la cohérence (un parent a toujours un chemin plus court), le tri par nom place naturellement les parents avant les enfants, et la structure se valide mécaniquement (chemins uniques, pas de trou). Inconvénient : moins lisible qu'un champ `parent` nommé, et inadapté dès que l'arbre n'est pas un arbre de position.

ATELIER — 30 minutes : trois hiérarchies, trois verdicts

Objectif : savoir choisir le bon mécanisme, et le prouver en code.

1. **(8 min)** Écrivez l'énumération `Gravite` (5.5) avec `passe(seuil)`. Affichez les deux tris (naturel et par nom) : constatez la divergence.
2. **(8 min)** Écrivez `Categorie` (5.2) avec `parent`, `profondeur`, `sous`. Affichez l'arbre indenté de la section 5.3.
3. **(8 min)** Écrivez `Code` (5.5, hiérarchie par préfixe). Ajoutez `INCIDENT_HORS_SERVICE` : que se passe-t-il au premier appel de `parent()` ? Corrigez par le champ explicite.
4. **(6 min)** Écrivez une hiérarchie scellée `Evenement` (5.4) avec trois `record`, et un `switch` exhaustif. Ajoutez un quatrième cas : le compilateur vous prévient-il ?

6 `switch` exhaustif et pattern matching

> Le `switch` sur énumération est l'endroit où Java a le plus progressé : les formes modernes vérifient l'exhaustivité à la compilation, ce qui transforme un oubli silencieux en erreur de build. Voici les formes, leurs règles exactes, et ce qui casse quand on ajoute une constante.

6.1 `switch` sur enum : la forme moderne

PROGRAMME — QUATRE FORMES, DU PLUS ANCIEN AU PLUS RÉCENT

```
enum Priorite { BASSE, NORMALE, HAUTE }
Priorite p = Priorite.HAUTE;

// 1. switch statement classique (Java 1.0)
switch (p) {
    case BASSE:
        System.out.println("basse");
        break;
    default:
        System.out.println("autre");
}

// 2. switch expression avec flèches (Java 14) — renvoie une valeur
String texte = switch (p) {
    case BASSE -> "priorité basse";
    case NORMALE -> "priorité normale";
    case HAUTE -> "priorité haute";
};

// 3. plusieurs constantes sur la même branche
String niveau = switch (p) {
    case BASSE, NORMALE -> "faible";
    case HAUTE -> { String s = "élevée"; yield s + " (urgence)"; }
};

// 4. switch statement moderne (flèches, pas de fall-through)
switch (p) {
    case HAUTE -> System.out.println("haut");
    default -> System.out.println("autre");
}
```

SORTIE RÉELLE

```
autre
switch ->>      : priorité haute
yield           : élevée (urgence)
default utile   : haut
```

La différence décisive entre les formes 1 et 2/4 : avec les flèches (`->`), il n'y a **pas de *fall-through***. Chaque branche est indépendante, aucun `break` n'est nécessaire, et l'oubli d'un `break` — source historique de bugs — devient impossible. De plus, la forme expression *doit* produire une valeur : si un cas manque, le compilateur refuse de compiler.

Ce que la JLS impose dans les `case`

Pour un `switch` sur énumération, l'étiquette d'un `case` doit être le **nom non qualifié** d'une constante : `case HAUTE`, jamais `case Priorite.HAUTE`. C'est une règle du langage, et elle a évolué : depuis Java 21, la forme qualifiée `case Priorite.HAUTE` est acceptée *dans les patterns* (voir 6.3), mais reste refusée dans un `switch` d'énumération classique :

REPRODUCTION — AVEC `--RELEASE 17`

```
Q.java:3: error: an enum switch case label must be the unqualified name
of an enumeration constant
    static String f(E e) { return switch (e) { case E.A -> "a"; case E.B -> "b"; }; }
```

6.2 `null`, `default`, et l'exhaustivité

Situation	Ce qui se passe	Que faire
`switch` sur une valeur `null`	`NullPointerException`, même si un `default` existe	Ajouter `case null ->` (6.3), ou tester `null` avant.
`switch` expression sans tous les cas	`error: the switch expression does not cover all possible input values`	Aucun correctif à chercher : c'est la protection. Traiter le cas manquant.
`switch` statement sans `default`	Compile. Le cas non traité est simplement ignoré.	Ajouter `default` avec une exception, ou passer en expression.
`switch` expression avec `default`	Compile, mais perd la vérification d'exhaustivité	À éviter : le `default` masque l'ajout d'une constante.

REPRODUCTION — `SWITCH` SUR `NULL`

```
Priorite p = null;
switch (p) {
    case BASSE -> System.out.println("basse");
    default -> System.out.println("autre");
}
```

SORTIE RÉELLE

```
switch(null)    : NullPointerException
```

Le `default` qui annule la protection du compilateur

C'est le piège le plus coûteux de tout ce chapitre. Un `switch` expression *sans* `default` ne compile plus si vous ajoutez une constante. Avec un `default`, il compile très bien — et votre nouvelle constante tombe silencieusement dans la branche par défaut, avec un comportement probablement faux. La règle : dans un `switch` d'énumération, **n'écrivez pas de `default`** si vous voulez être prévenu. Si vous en avez besoin pour satisfaire le compilateur, mettez `default -> throw new IllegalStateException("cas non traité : " + p);` : bruyant, donc corrigeable.

6.3 Patterns : `case ROUGE`, `case null`, `when`

Depuis Java 21, les patterns peuvent apparaître dans un `switch` sur un type quelconque — et une constante d'énumération y est un *pattern de constante*. Cela permet des choses impossibles avant : tester une constante sur une variable `Object`, capturer la valeur, ou ajouter une garde.

PROGRAMME

```
// a) pattern de constante sur un Object
Object o = Priorite.HAUTE;
String r1 = switch (o) {
    case Priorite.BASSE -> "c'était BASSE";
    case Priorite p -> "priorité " + p;      // pattern de type + nom
    case null -> "null";
    default -> "autre";
};

// b) case null explicite (évite le NPE)
Priorite nul = null;
String r2 = switch (nul) {
    case null -> "aucune";
    case BASSE -> "b";
    default -> "?";
};

// c) garde « when » sur un record qui contient une énumération
record Incident(Priorite priorite, int duree) { }
Object x = new Incident(Priorite.HAUTE, 30);
String r3 = switch (x) {
    case Incident(Priorite pr, int d) when pr == Priorite.HAUTE && d < 60 ->
            "urgence courte (" + d + " min)";
    case Incident(Priorite pr, int d) -> "incident " + pr + " de " + d + " min";
    default -> "autre";
};
```

SORTIE RÉELLE

```
pattern constant: priorité HAUTE
case null       : aucune
record pattern  : urgence courte (30 min)
```

> **Ordre des cas : le compilateur détecte les cas dominés**
>
> Avec les patterns, l'ordre compte : un `case Priorite p` placé avant `case Priorite.BASSE` rendrait ce dernier inaccessible. `javac` signale alors `error: this case label is dominated by a preceding case label`. C'est une erreur de compilation, pas un avertissement — bonne nouvelle. Placez toujours les cas les plus spécifiques en premier.

6.4 Ajouter une constante : ce qui casse, ce qui ne casse pas

Voici l'expérience la plus utile du chapitre. Nous avons une énumération `Statut` et un `switch` expression qui la couvre entièrement. Ajoutons une constante `ANNULEE`, et regardons ce qui se passe — successivement sur les trois formes.

Forme utilisée	Après ajout de `ANNULEE`	Verdict
`switch` expression sans `default`	`error: the switch expression does not cover all possible input values`	**Idéal** : impossible d'oublier.
`switch` statement avec flèches, sans `default`	Compile. Rien ne se passe pour `ANNULEE`.	Dangereux : silence total.
`switch` statement classique avec `default`	Compile ; `ANNULEE` tombe dans le `default`.	Acceptable si le `default` lève une exception.
`switch` expression avec `default`	Compile ; `ANNULEE` prend la valeur par défaut.	Dangereux : comportement faux et silencieux.

REPRODUCTION EXACTE

```
// 1. la version qui couvre tous les cas
static String libelle(Statut s) {
    return switch (s) {
        case CREE -> "créée";
        case PAYEE -> "payée";
        case EXPEDIEE -> "expédiée";
    };
}

// 2. on ajoute une constante à l'enum
enum Statut { CREE, PAYEE, EXPEDIEE, ANNULEE }

// 3. on recompile
```

SORTIE RÉELLE

```
Exh.java:7: error: the switch expression does not cover all possible input values
        return switch (s) {
               ^
1 error
```

ET LA VERSION « ANCIEN STYLE », QUI NE DIT RIEN

```
switch (s) {
    case CREE: System.out.println("créée"); break;
    case PAYEE: System.out.println("payée"); break;
}
System.out.println("aucune erreur : le cas manquant est simplement ignoré");
```

SORTIE RÉELLE

```
payée
aucune erreur : le cas manquant est simplement ignoré
```

La règle de conduite, applicable dès aujourd'hui

Partout où vous écrivez du `switch` sur une énumération, utilisez la forme **expression** (avec `->` et une valeur de retour), et n'ajoutez de `default` que s'il lève une exception explicite. Vous transformez ainsi chaque ajout de constante en erreur de compilation — c'est-à-dire en une liste de tâches automatique, générée par le compilateur. Sur un projet vivant, c'est ce qui évite les « j'ai ajouté le statut mais l'export CSV ne le connaît pas ».

7 Cas pratiques vérifiés

> Cinq situations de projet, résolues avec les outils des chapitres précédents. Chaque exemple est court, complet et exécutable tel quel.

7.1 Un cycle de vie avec transitions autorisées

Déjà esquissé en 2.3, voici la version complète : validation, test, et une méthode `changer` qui refuse les transitions interdites.

FICHIER — `STATUT.JAVA`

```
public enum Statut {
    OUVERT, EN_COURS, RESOLU, FERME;

    public boolean peutAllerVers(Statut suivant) {
        return switch (this) {
            case OUVERT    -> suivant == EN_COURS || suivant == FERME;
            case EN_COURS  -> suivant == RESOLU   || suivant == FERME;
            case RESOLU    -> suivant == EN_COURS || suivant == FERME;
            case FERME     -> false;
        };
    }

    public Statut vers(Statut suivant) {
        if (!peutAllerVers(suivant)) {
            throw new IllegalStateException("Transition interdite : " + this + " -> " + suivant);
        }
        return suivant;
    }
}
```

PROGRAMME

```
System.out.println(Statut.OUVERT.vers(Statut.EN_COURS));
System.out.println(Statut.EN_COURS.vers(Statut.RESOLU));
try {
    Statut.FERME.vers(Statut.OUVERT);
} catch (IllegalStateException e) {
    System.out.println("refus : " + e.getMessage());
}
```

SORTIE RÉELLE

```
EN_COURS
RESOLU
refus : Transition interdite : FERME -> OUVERT
```

7.2 Regrouper et compter avec `EnumMap`

PROGRAMME — TROIS FAÇONS DE COMPTER, DONT LA BONNE

```
List<Tache> taches = List.of(
        new Tache("Rapport", Priorite.BASSE, 120),
        new Tache("Incident", Priorite.URGENTE, 30),
        new Tache("Revue de code", Priorite.HAUTE, 45),
        new Tache("Sauvegarde", Priorite.NORMALE, 15),
        new Tache("Audit", Priorite.HAUTE, 45));

// 1. avec valeurs à zéro : toutes les constantes sont présentes
EnumMap<Priorite, Long> stats = new EnumMap<>(Priorite.class);
for (Priorite p : Priorite.values()) stats.put(p, 0L);
for (Tache t : taches) stats.merge(t.priorite(), 1L, Long::sum);
System.out.println("avec zéros     : " + stats);

// 2. avec un collecteur et une fabrique EnumMap
Map<Priorite, List<Tache>> groupes = taches.stream()
        .collect(Collectors.groupingBy(Tache::priorite,
                () -> new EnumMap<>(Priorite.class), Collectors.toList()));
System.out.println("groupingBy     : " + groupes);

// 3. ce qu'il ne faut pas faire : HashMap
Map<Priorite, Long> hash = new HashMap<>();
for (Tache t : taches) hash.merge(t.priorite(), 1L, Long::sum);
System.out.println("HashMap        : " + hash + "  (ordre non garanti)");
```

SORTIE RÉELLE

```
avec zéros     : {BASSE=1, NORMALE=1, HAUTE=2, URGENTE=1}
groupingBy     : {BASSE=[Rapport (BASSE, 120 min)], NORMALE=[Sauvegarde (NORMALE, 15 min)],
HAUTE=[Revue de code (HAUTE, 45 min), Audit (HAUTE, 45 min)], URGENTE=[Incident (URGENTE, 30 min)]}
HashMap        : {HAUTE=2, BASSE=1, NORMALE=1, URGENTE=1}  (ordre non garanti)
```

Pourquoi préremplir l'`EnumMap` avec des zéros

Un tableau de bord doit afficher toutes les catégories, y compris celles à zéro. Si vous ne préremplissez pas, la clé absente oblige l'affichage à gérer `null` — et le jour où vous oubliez, vous obtenez une `NullPointerException` sur un `Long` déballé. Deux lignes de préremplissage suppriment toute une classe de bugs. Alternative élégante : `taches.stream().collect(Collectors.groupingBy(Tache::priorite, () -> new EnumMap<>(Priorite.class), Collectors.counting()))`, puis compléter — mais le préremplissage explicite reste le plus lisible.

7.3 Persister : toujours `name()`, jamais `ordinal()`

C'est le piège qui coûte le plus cher en production, parce qu'il est **silencieux**. Nous l'avons reproduit : une première version de l'application écrit des données avec trois constantes ; on ajoute `CRITIQUE` entre `BASSE` et `NORMALE` ; on relit les données avec la nouvelle version.

PROGRAMME — ÉCRITURE AVEC LA VERSION 1

```
// enum Priorite { BASSE, NORMALE, HAUTE }
try (PrintWriter w = new PrintWriter("donnees.txt")) {
    for (Priorite p : Priorite.values()) {
        w.println(p.name() + ";" + p.ordinal());
    }
}
```

CONTENU PRODUIT

```
BASSE;0
NORMALE;1
HAUTE;2
```

PROGRAMME — RELECTURE AVEC LA VERSION 2 (`CRITIQUE` INSÉRÉ)

```
// enum Priorite { BASSE, CRITIQUE, NORMALE, HAUTE }
for (String ligne : Files.readAllLines(Path.of("donnees.txt"))) {
    String[] p = ligne.split(";");
    int ordinal = Integer.parseInt(p[1]);
    String parOrdinal = Priorite.values()[ordinal].name();
    String parNom = Priorite.valueOf(p[0]).name();
    System.out.printf("stocké %-8s ordinal=%d -> par ordinal: %-9s | par nom: %-9s%n",
            p[0], ordinal, parOrdinal, parNom);
}
```

SORTIE RÉELLE

```
stocké BASSE    ordinal=0 -> par ordinal: BASSE     | par nom: BASSE
stocké NORMALE  ordinal=1 -> par ordinal: CRITIQUE  | par nom: NORMALE     <-- FAUX
stocké HAUTE    ordinal=2 -> par ordinal: NORMALE   | par nom: HAUTE
```

Le bilan : des données corrompues, sans aucune erreur

Aucune exception, aucun avertissement : toutes les lignes « NORMALE » de la base sont devenues des incidents « CRITIQUE », et toutes les « HAUTE » sont devenues des « NORMALE ». Sur 300 000 lignes, c'est une perte de données silencieuse. **Les trois règles :**

1. Persistez `name()` (ou un code explicite et stable), **jamais** `ordinal()`.
2. Ne renommez jamais une constante persistée : si vous devez le faire, prévoyez une migration (`UPDATE … SET statut='NOUVEAU' WHERE statut='ANCIEN'`), ou une constante d'alias dans `valueOf`.
3. Ajoutez un test qui échoue si l'on retire ou renomme une constante : `assertEquals(Set.of("BASSE","NORMALE","HAUTE"), Arrays.stream(Priorite.values()).map(Enum::name).collect(toSet()))`. Ce test casse au moment du renommage, avec un message clair, plutôt qu'en production six mois plus tard.

7.4 Mesures : `values()`, `EnumSet`, `EnumMap`

Trois mesures réelles sur cette machine (Temurin 25.0.5, Debian 13, 1 vCPU), avec échauffement et meilleur temps sur cinq passes. Les chiffres sont indicatifs : retenez les ordres de grandeur et le *sens* des écarts.

Comment ces chiffres ont été obtenus (et pourquoi ça compte)

Micro-benchmark écrit à la main : échauffement de 5 passes, puis meilleur temps sur 5 passes, avec une variable globale qui consomme les résultats pour empêcher la JIT de supprimer la boucle. Ce n'est **pas** un benchmark rigoureux (il faudrait JMH, qui gère la compilation à la volée, les transitions de pile et le biais de mesure). Les chiffres suffisent à comparer des ordres de grandeur ; ils ne doivent pas servir à trancher un choix d'architecture. Sur un cas réel, mesurez votre propre code avec JMH.

Opération (2 millions d'itérations)	Temps	Commentaire
`values()` à chaque itération	≈ 6 807 µs	Un tableau cloné par appel
Tableau de `values()` mis en cache	≈ 959 µs	**7× plus rapide**
`EnumSet` : `add` + `contains`	≈ 7 385 µs	Masque de bits
`HashSet` : `add` + `contains`	≈ 10 824 µs	Hachage + table

Ce qu'il faut retenir de ces chiffres (et ce qu'il ne faut pas en conclure)

À retenir : `values()` coûte une allocation par appel — sortez-le des boucles chaudes ; `EnumSet` est plus rapide que `HashSet` et surtout *ordonné*. **À ne pas conclure :** que `EnumMap` est toujours plus rapide que `HashMap`. Nos mesures donnent 3 à 4 ms dans les deux sens selon la passe, sans écart significatif sur 1 million d'opérations — la JIT optimise très bien les deux. Le vrai argument pour `EnumMap` est l'**ordre déterministe** et l'absence de collision, pas la vitesse brute. Ne réécrivez pas votre code pour 2 ms : réécrivez-le pour la lisibilité et la reproductibilité.

7.5 Tester une énumération avec JUnit 5

FICHIER — `PRIORITETEST.JAVA`

```
import static org.junit.jupiter.api.Assertions.*;

import java.util.Arrays;
import java.util.Set;
import java.util.stream.Collectors;

import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.EnumSource;

class PrioriteTest {

    @Test
    void ordre_naturel_va_du_moins_au_plus_prioritaire() {
        assertTrue(Priorite.BASSE.compareTo(Priorite.URGENTE) < 0);
        assertArrayEquals(
                new Priorite[] { Priorite.BASSE, Priorite.NORMALE, Priorite.HAUTE,
Priorite.URGENTE },
                Priorite.values());
    }

    @Test
    void les_noms_sont_stables() {
        // échoue si une constante est renommée ou supprimée : protège la persistance
        Set<String> noms =
Arrays.stream(Priorite.values()).map(Enum::name).collect(Collectors.toSet());
        assertEquals(Set.of("BASSE", "NORMALE", "HAUTE", "URGENTE"), noms);
    }

    @Test
    void valueOf_est_sensible_a_la_casse() {
        assertThrows(IllegalArgumentException.class, () -> Priorite.valueOf("Haute"));
    }

    @ParameterizedTest
    @EnumSource(Priorite.class)
    void chaque_constante_est_coherente(Priorite p) {
        assertEquals(p.name(), p.toString());
        assertTrue(p.poids() >= 1 && p.poids() <= 4);
        assertSame(p, Priorite.valueOf(p.name()));
    }

    @Test
    void les_transitions_interdites_levent_une_exception() {
        assertThrows(IllegalStateException.class, () -> Statut.FERME.vers(Statut.OUVERT));
        assertEquals(Statut.EN_COURS, Statut.OUVERT.vers(Statut.EN_COURS));
    }
}
```

Le test qui vaut tous les autres : `@EnumSource`

`@ParameterizedTest` + `@EnumSource(MaClasse.class)` exécute automatiquement le test pour **chaque constante**, y compris celles que vous ajouterez plus tard. C'est la seule façon d'être certain qu'une nouvelle constante respecte les invariants (nom non nul, poids dans les bornes, `valueOf(name())` identique). Associé au test de stabilité des noms, il couvre les deux risques majeurs : régression de comportement et rupture de persistance.

LANCEMENT (JUNIT PLATFORM CONSOLE, SANS BUILD TOOL)

```
javac --release 21 -cp "lib/*:out" -d test-out $(find src/test/java -name '*.java')
java -jar lib/junit-platform-console-standalone-1.12.2.jar execute \
     --class-path "out:test-out" --scan-class-path --details=tree
```

SORTIE RÉELLE (EXTRAIT AVEC `--DETAILS=TREE`)

```
│     ├─ les_transitions_interdites_levent_une_exception() ✔
│     ├─ chaque_constante_est_coherente(Priorite) ✔
│     │  ├─ [1] BASSE ✔
│     │  ├─ [2] NORMALE ✔
│     │  ├─ [3] HAUTE ✔
│     │  └─ [4] URGENTE ✔
│     ├─ les_noms_sont_stables() ✔
│     ├─ valueOf_est_sensible_a_la_casse() ✔
│     └─ ordre_naturel_va_du_moins_au_plus_prioritaire() ✔

[         8 tests found           ]
[         8 tests successful      ]
[         0 tests failed          ]
```

8 Erreurs et pièges : 12 cas réels

> Douze erreurs, toutes reproduites sur la machine qui a servi à écrire ce document. Les messages sont recopiés tels quels. Trois familles : ce qui ne compile pas (facile), ce qui compile mais se comporte mal (coûteux), et ce qui ne se voit qu'à l'exécution ou en production (dangereux).

N°	Symptôme	Cause et correctif
1	`UnsupportedOperationException` au tri	La liste vient de `List.of` : immuable. → `new ArrayList<>(…)` ou `stream().sorted()`.
2	`NullPointerException` pendant `list.sort(null)`	Un élément vaut `null`. → `Comparator.nullsLast(...)`.
3	Tri « alphabétique » faux : `ETAPE_10` avant `ETAPE_2`	`String.compareTo` est lexicographique. → rembourrer (`ETAPE_02`) ou comparateur naturel.
4	Tri avec `É` à la fin	`É` > `Z` en UTF-16. → `Collator.getInstance(Locale.FRENCH)`.
5	`No enum constant … Haute`	`valueOf` est sensible à la casse et aux espaces. → `equalsIgnoreCase` + `trim()` (section 2.4).
6	Données corrompues après un ajout de constante	`ordinal()` persisté. → persister `name()` (section 7.3).
7	`IllegalArgumentException : HAUTE > BASSE`	`EnumSet.range(a, b)` exige `a <= b`. → inverser, ou tester `ordinal()`.
8	`NullPointerException` dans `switch`	Valeur `null`. → `case null ->` ou test préalable.
9	`ClassCastException` sur `compareTo`	Deux enum différents, ou une valeur d'un autre type. → corriger le type.
10	`ClassCastException` dans `TreeSet` avec comparateur `reversed()`	`TreeSet(comparator)` exige un comparateur cohérent, non `null`. Le piège est de passer un `null` comparateur en croyant trier naturellement.
11	Deux tâches « égales » disparaissent d'un `TreeSet`	Comparateur ne comparant qu'un champ : l'arbre considère les éléments identiques. → ajouter un critère de départage.
12	`switch` qui compile mais ignore une constante	`default` dans un `switch` expression. → retirer le `default` ou le faire lever.

8.1 Ce qui ne compile pas (et c'est une bonne nouvelle)

ERREUR — CONSTANTE NON COUVERTE PAR UN `SWITCH` EXPRESSION

```
Exh.java:7: error: the switch expression does not cover all possible input values
        return switch (s) {
               ^
1 error
```

ERREUR — `CASE` QUALIFIÉ DANS UN SWITCH D'ÉNUMÉRATION

```
Q.java:3: error: an enum switch case label must be the unqualified name
of an enumeration constant
    static String f(E e) { return switch (e) { case E.A -> "a"; case E.B -> "b"; }; }
```

ERREUR — TENTATIVE DE REDÉFINIR `COMPARETO`

```
public class T { enum E implements Comparable<E> { A, B;
    public int compareTo(E o){ return 0; } } }
```

MESSAGE RÉEL

```
T.java:1: error: compareTo(T.E) in T.E cannot override compareTo(E) in Enum
  overridden method is final
1 error
```

Pourquoi ces trois-là sont les erreurs « confortables »

Elles vous empêchent de livrer. Elles arrivent immédiatement, avant toute exécution, et le message pointe la ligne exacte. Le vrai danger des énumérations est ailleurs : dans les erreurs 5 à 12, qui compilent parfaitement et se manifestent en production. C'est la raison d'être des tests de la section 7.5.

8.2 Ce qui compile mais se comporte mal

Erreur 1 — trier une liste immuable

REPRODUCTION

```
List<Priorite> l = List.of(Priorite.HAUTE, Priorite.BASSE);
l.sort(null);
```

SORTIE RÉELLE

```
Exception in thread "main" java.lang.UnsupportedOperationException
        at java.base/java.util.ImmutableCollections.uoe(ImmutableCollections.java:142)
```

Correctif : `new ArrayList<>(List.of(…))` si vous voulez modifier, ou `List.of(…).stream().sorted().toList()` si vous voulez une nouvelle liste triée. Notez que `Arrays.asList` est modifiable en partie seulement : `set` fonctionne, `add` non. Une seule forme est sûre pour les tris en place : `new ArrayList<>(…)`.

Erreur 7 — `EnumSet.range` à l'envers

REPRODUCTION

```
EnumSet.range(Priorite.HAUTE, Priorite.BASSE);
```

SORTIE RÉELLE

```
Exception in thread "main" java.lang.IllegalArgumentException: HAUTE > BASSE
```

Cause : `range` suppose que la première borne précède la seconde dans l'ordre de déclaration. Le message est très clair (il affiche les deux constantes et la comparaison), ce qui est rare — d'où l'intérêt de le connaître. **Correctif :** inverser les bornes, ou construire l'ensemble par `EnumSet.copyOf` / `of` si l'ordre de déclaration n'est pas celui que vous voulez.

Erreurs 3 et 4 — deux tris qui « marchent » mais mentent

RAPPEL DES SORTIES RÉELLES

```
par name() : [ETAPE_1, ETAPE_10, ETAPE_11, ETAPE_2, ETAPE_3]
compareTo (UTF-16) : [EAU, ELEVE, ETE, ZEBRE, ÉTÉ]
```

Aucune exception, aucune erreur, et pourtant l'affichage est faux du point de vue de l'utilisateur. **Correctifs :** rembourrage des numéros (`ETAPE_01`), comparateur alphanumérique, ou `Collator`. Détail complet au chapitre 4.4.

Erreur 11 — l'élément qui disparaît d'un `TreeSet`

REPRODUCTION

```
record Tache(String nom, Priorite priorite) { }
Comparator<Tache> partiel = Comparator.comparingInt(t -> t.dureeMinutes());  // un seul critère

Set<Tache> ensemble = new TreeSet<>(partiel);
ensemble.add(new Tache("Audit", Priorite.HAUTE, 45));
ensemble.add(new Tache("Revue", Priorite.HAUTE, 45));
System.out.println(ensemble.size());   // 1, pas 2
```

Cause : un `TreeSet` élimine les doublons *selon son comparateur*, pas selon `equals`. Deux tâches de même durée sont donc « égales » pour l'ensemble, et la seconde est rejetée silencieusement. **Correctif :** un comparateur d'ordre total, avec un dernier critère discriminant (`thenComparing(Tache::nom)`), exactement comme `ORDRE_METIER` au chapitre 4.6.

> **La règle « comparateur cohérent avec equals »**
>
> La Javadoc de `Comparator` l'exige pour les structures triées : si `compare(a, b) == 0`, alors `a.equals(b)` doit être vrai. Un comparateur partiel viole ce contrat, et le résultat dépend de la structure : `List.sort` accepte (deux éléments « égaux » restent tous les deux, dans un ordre arbitraire), `TreeSet` et `TreeMap` refusent (ils perdent des éléments). C'est l'erreur la plus difficile à diagnostiquer du chapitre, parce que rien ne la signale.

8.3 Ce qui se voit en production

Erreur 6 — la corruption silencieuse par `ordinal()`

RAPPEL DE LA SORTIE RÉELLE (CHAPITRE 7.3)

```
stocké BASSE    ordinal=0 -> par ordinal: BASSE     | par nom: BASSE
stocké NORMALE  ordinal=1 -> par ordinal: CRITIQUE  | par nom: NORMALE     <-- FAUX
stocké HAUTE    ordinal=2 -> par ordinal: NORMALE   | par nom: HAUTE
```

C'est **l'erreur la plus coûteuse** de tout ce document : aucune exception, aucune trace, et des données fausses. Le correctif est unique et sans compromis : persistez `name()`. Ajoutez un test de stabilité des noms (section 7.5) pour être prévenu dès le renommage.

Erreur 8 — le `switch` sur `null`

REPRODUCTION

```
Priorite p = null;
switch (p) {
    case BASSE -> System.out.println("basse");
    default -> System.out.println("autre");
}
```

SORTIE RÉELLE

```
switch(null)    : NullPointerException
```

Subtilité : le `default` ne protège pas du `null`. La JVM évalue le sélecteur du `switch` avant tout : un `null` lève une exception immédiatement. **Correctif :** `case null -> "aucune"` (chapitre 6.3), ou un test

`if (p == null)` en amont. Attention : `case null` n'est autorisé que dans un `switch` qui traite aussi des patterns ou des constantes — il fonctionne dans les deux cas depuis Java 21.

Erreur 5 — `valueOf` sur une saisie utilisateur

REPRODUCTION

```
Priorite.valueOf("Haute");    // casse différente
Priorite.valueOf(" HAUTE");   // espace en tête
Priorite.valueOf("");         // chaîne vide
```

SORTIES RÉELLES

```
No enum constant fr.nice.enums.Priorite.Haute
No enum constant fr.nice.enums.Priorite. HAUTE
No enum constant fr.nice.enums.Priorite.
```

Cause : `valueOf` fait une recherche exacte dans le tableau des noms. Il n'y a ni normalisation, ni tolérance. **Correctif :** une méthode maison qui normalise (section 2.4), et une `IllegalArgumentException` avec un message utilisable — celle de `valueOf` affiche le nom du type et la chaîne reçue, ce qui est correct en développement mais insuffisant dans une interface utilisateur.

9 Fiche de synthèse et aide-mémoire

> Deux pages à garder à côté de l'éditeur. Toutes les lignes ont été exécutées dans ce document.

9.1 Créer

AIDE-MÉMOIRE

```
// minimale
enum Couleur { ROUGE, VERT }

// avec champs (constantes d'abord, PUIS point-virgule, puis les membres)
enum Priorite {
    BASSE(1), NORMALE(2), HAUTE(3), URGENTE(4);
    private final int poids;
    Priorite(int poids) { this.poids = poids; }
    public int poids() { return poids; }
}

// avec comportement par constante (méthode abstraite)
enum Taille { S(1) { String cat() { return "petite"; } }, L(4) { String cat() { return
"grande"; } };
    private final int rang; Taille(int r) { rang = r; } abstract String cat(); }

// qui implémente une interface
enum Niveau implements Etiquette { INFO, ERREUR; public String etiquette(String m) { return "[" +
name() + "] " + m; } }

// avec recherche par code (cache déclaré APRÈS les constantes)
private static final Map<String, Devise> PAR_CODE =
    Stream.of(values()).collect(Collectors.toUnmodifiableMap(Devise::codeNumerique,
Function.identity()));
```

9.2 Comparer

AIDE-MÉMOIRE

```
// identité (toujours préféré)
if (p == Priorite.HAUTE) { … }

// ordre naturel = ordre de déclaration
int c = Priorite.HAUTE.compareTo(Priorite.BASSE);      // > 0 car HAUTE est déclarée après
boolean auMoins = p.compareTo(seuil) >= 0;

// avec null possible
Comparator<Priorite> sur = Comparator.nullsLast(Comparator.naturalOrder());
boolean egal = Objects.equals(p, Priorite.HAUTE);

// ne pas confondre
p.name()      // "HAUTE"  — stable, pour la machine
p.ordinal()   // 2        — instable, pour EnumSet/EnumMap uniquement
p.toString()  // "HAUTE" ou un libellé si redéfini — pour l'utilisateur
```

9.3 Trier

AIDE-MÉMOIRE

```
// ordre naturel (déclaration)
liste.sort(null);
Collections.sort(liste);
liste.stream().sorted().toList();

// ordre inverse
liste.sort(Comparator.reverseOrder());
liste.sort(Collections.reverseOrder());

// par nom (alphabétique)
liste.sort(Comparator.comparing(Priorite::name));
liste.sort(Comparator.comparing(Enum::name).reversed());

// par attribut
liste.sort(Comparator.comparingInt(Priorite::poids));
liste.sort(Comparator.comparing(Priorite::poids).reversed());

// multi-critères
liste.sort(Comparator.comparing(Tache::priorite).reversed()
                    .thenComparingInt(Tache::dureeMinutes)
                    .thenComparing(Tache::nom));

// avec null
liste.sort(Comparator.comparing(Fiche::priorite,
Comparator.nullsLast(Comparator.naturalOrder())));

// linguistique (accents, casse)
Collator fr = Collator.getInstance(Locale.FRENCH);
liste.sort(Comparator.comparing(Enum::name, fr));
liste.sort(Comparator.comparing(Enum::name, String.CASE_INSENSITIVE_ORDER));

// structures ordonnées
EnumSet.allOf(Priorite.class)          // ordre de déclaration
EnumSet.range(BASSE, NORMALE)          // bornes incluses
EnumSet.complementOf(EnumSet.of(HAUTE))
EnumSet.copyOf(liste)
new EnumMap<>(Priorite.class)          // clés triées par déclaration
new TreeSet<>(liste)
new TreeSet<>(Comparator.reverseOrder())
```

9.4 Hiérarchiser

AIDE-MÉMOIRE

```
// 1. hiérarchie implicite : l'ordre de déclaration
//    déclarer du plus petit au plus grand, et le documenter dans la Javadoc

// 2. hiérarchie explicite : un champ parent (déclarer les parents AVANT les enfants)
enum Categorie {
    INCIDENT(null), RESEAU(INCIDENT), DNS(RESEAU);
    private final Categorie parent;
    Categorie(Categorie parent) { this.parent = parent; }
    public Categorie parent() { return parent; }
    public boolean estRacine() { return parent == null; }
    public int profondeur() { int d = 0; for (Categorie c = parent; c != null; c = c.parent) d++;
return d; }
    public boolean sous(Categorie a) { for (Categorie c = this; c != null; c = c.parent) if (c ==
a) return true; return false; }
}

// 3. enfants, groupés par parent
EnumMap<Categorie, List<Categorie>> enfants = new EnumMap<>(Categorie.class);
for (Categorie c : Categorie.values())
    if (c.parent() != null) enfants.computeIfAbsent(c.parent(), k -> new ArrayList<>()).add(c);

// 4. tri par niveau
liste.sort(Comparator.comparingInt(Categorie::profondeur).thenComparing(Categorie::name));

// 5. hiérarchie de TYPES (données différentes par cas) : sealed + record
sealed interface Evenement permits Creation, Suppression {}
record Creation(String nom) implements Evenement {}
record Suppression(String nom, String motif) implements Evenement {}
```

9.5 Les dix règles à retenir

N°	Règle
1	Les constantes viennent **toujours en premier** ; le point-virgule est obligatoire dès qu'un membre suit.
2	L'ordre de déclaration **est** l'ordre naturel. Le documenter dans la Javadoc si l'enum est publique.
3	Comparer avec `==`, jamais avec `equals` (inutile) ni avec `compareTo == 0` (détourné).
4	Pour trier par nom : `Comparator.comparing(E::name)`. Il n'y a pas de tri par nom implicite.
5	Trier sur un **attribut métier**, pas sur `ordinal()`, quand l'ordre compte vraiment.
6	`EnumSet` et `EnumMap` plutôt que `HashSet`/`HashMap` : ordre garanti et déterminisme.
7	Persister `name()`, **jamais** `ordinal()`. Tester la stabilité des noms.
8	`switch` expression sans `default` : le compilateur devient votre liste de tâches.
9	Une hiérarchie se code avec un champ `parent` déclaré de haut en bas, jamais par héritage.
10	`values()` alloue un tableau : le sortir des boucles chaudes.

10 Exercices corrigés

> Huit exercices, du plus simple au plus complet, avec les solutions et les commandes de vérification. Chacun tient dans un fichier ; les corrigés donnent la sortie attendue pour que vous puissiez comparer.

N°	Énoncé	Points	Compétence
1	Écrire une énumération avec champs et méthode	10	Constructeur, champs, `name`
2	Trier par ordinal, par nom, par attribut	15	`Comparator`
3	Corriger un tri alphabétique faux	10	Nombres et accents
4	Construire une hiérarchie à trois niveaux	20	Champ `parent`
5	Compter et regrouper avec `EnumMap`	15	Collecteurs
6	Rendre un `switch` exhaustif et sûr	15	Switch expression
7	Persister sans se tromper	15	`name()` contre `ordinal()`
8	Choisir entre enum, hiérarchie scellée et table	10	Conception

10.1 Exercice 1 — Une énumération avec champs (10 points)

Énoncé. Écrivez une énumération `Planete` contenant les huit planètes du système solaire, chacune avec sa masse relative à la Terre (en `double`) et son nombre de satellites connus. Ajoutez une méthode `poidsSurTerre(double kg)` qui renvoie le poids d'un objet sur cette planète, et une méthode `plusLourdeQue(Planete)`. L'ordre de déclaration doit aller du Soleil vers l'extérieur.

CORRIGÉ

```
public enum Planete {
    MERCURE(0.055, 0),
    VENUS(0.815, 0),
    TERRE(1.0, 1),
    MARS(0.107, 2),
    JUPITER(317.8, 95),
    SATURNE(95.2, 274),
    URANUS(14.5, 28),
    NEPTUNE(17.1, 16);

    private final double masseRelative;
    private final int satellites;

    Planete(double masseRelative, int satellites) {
        this.masseRelative = masseRelative;
        this.satellites = satellites;
    }

    public double masseRelative() { return masseRelative; }

    public int satellites() { return satellites; }

    /** Poids d'un objet de {@code masseKg} kilogrammes à la surface de cette planète. */
    public double poidsSurTerre(double masseKg) {
        return masseKg * masseRelative;
    }

    public boolean plusLourdeQue(Planete autre) {
        return this.masseRelative > autre.masseRelative;
    }
}
```

VÉRIFICATION

```
System.out.println(Planete.MARS.poidsSurTerre(70));
System.out.println(Planete.JUPITER.plusLourdeQue(Planete.TERRE));
System.out.println(Arrays.toString(Planete.values()));
```

SORTIE ATTENDUE

```
7.49
true
[MERCURE, VENUS, TERRE, MARS, JUPITER, SATURNE, URANUS, NEPTUNE]
```

Barème : huit constantes avec arguments (3 pts) · champs `final` et constructeur (3 pts) · `poidsSurTerre` correcte (2 pts) · `plusLourdeQue` utilisant l'attribut et non l'`ordinal()` (2 pts).

10.2 Exercice 2 — Les trois tris (15 points)

Énoncé. Avec `Planete`, affichez trois listes : (a) dans l'ordre de déclaration, (b) par ordre alphabétique du nom, (c) par masse croissante. Puis affichez la planète la plus lourde et la plus légère.

CORRIGÉ

```
List<Planete> base = new ArrayList<>(List.of(Planete.values()));

List<Planete> a = new ArrayList<>(base);
a.sort(null);

List<Planete> b = new ArrayList<>(base);
b.sort(Comparator.comparing(Planete::name));

List<Planete> c = new ArrayList<>(base);
c.sort(Comparator.comparingDouble(Planete::masseRelative));

System.out.println("déclaration : " + a);
System.out.println("par nom     : " + b);
System.out.println("par masse   : " + c);
System.out.println("max masse   : " + Collections.max(base,
Comparator.comparingDouble(Planete::masseRelative)));
System.out.println("min masse   : " + Collections.min(base,
Comparator.comparingDouble(Planete::masseRelative)));
```

SORTIE ATTENDUE

```
déclaration : [MERCURE, VENUS, TERRE, MARS, JUPITER, SATURNE, URANUS, NEPTUNE]
par nom     : [JUPITER, MARS, MERCURE, NEPTUNE, SATURNE, TERRE, URANUS, VENUS]
par masse   : [MERCURE, MARS, VENUS, TERRE, URANUS, NEPTUNE, SATURNE, JUPITER]
max masse   : JUPITER
min masse   : MERCURE
```

Barème : trois copies indépendantes de la liste — et non un tri en place réutilisé (5 pts) · `comparing` et `comparingDouble` corrects (5 pts) · `max`/`min` avec comparateur (5 pts).

> **L'erreur attendue**
>
> Réutiliser la même liste pour les trois tris : le deuxième tri part d'une liste déjà triée, donc l'affichage ne démontre rien — et si vous utilisez `List.of`, vous obtenez `UnsupportedOperationException`. Trois `new ArrayList<>(base)` valent mieux qu'un commentaire.

10.3 Exercice 3 — Corriger un tri faux (10 points)

Énoncé. On vous donne cette énumération et ce tri. Le résultat affiché est `[SPRINT_1, SPRINT_10, SPRINT_11, SPRINT_2]`, ce qui est jugé incorrect par le client. Corrigez *sans* changer les noms des constantes (elles sont persistées en base).

CODE FOURNI

```
enum Sprint { SPRINT_1, SPRINT_2, SPRINT_10, SPRINT_11 }

List<Sprint> l = new ArrayList<>(List.of(Sprint.values()));
l.sort(Comparator.comparing(Sprint::name));
System.out.println(l);
```

CORRIGÉ — EXTRACTION DU NUMÉRO, SANS RENOMMER

```
static int numero(Sprint s) {
    return Integer.parseInt(s.name().substring("SPRINT_".length()));
}

List<Sprint> l = new ArrayList<>(List.of(Sprint.values()));
l.sort(Comparator.comparingInt(Sprint::numero));
System.out.println(l);

// variante : attribut porté par la constante (la plus propre)
// SPRINT_1(1), SPRINT_2(2), SPRINT_10(10), SPRINT_11(11);
// puis Comparator.comparingInt(Sprint::numero)
```

SORTIE ATTENDUE

```
[SPRINT_1, SPRINT_2, SPRINT_10, SPRINT_11]
```

Barème : tri numérique et non lexicographique (6 pts) · noms de constantes inchangés (2 pts) · solution robuste, idéalement un attribut `numero` plutôt qu'une analyse de chaîne (2 pts).

> **La leçon de conception**
>
> Si vous aviez la main sur les noms, `SPRINT_01` aurait suffi. Quand ce n'est pas le cas — et c'est le cas dès qu'une donnée est persistée — l'ordre doit venir d'un **attribut**, jamais d'une analyse de chaîne dans le code appelant. L'analyse de chaîne casse dès qu'un nom ne respecte pas exactement le format.

10.4 Exercice 4 — Une hiérarchie à trois niveaux (20 points)

Énoncé. Construisez une énumération `Rayon` représentant un classement de produits : deux racines (`ALIMENTAIRE`, `MAISON`), trois enfants de la première, deux de la seconde, et deux petits-enfants sous un enfant. Fournissez `parent()`, `profondeur()`, `chemin()`, `sous(Rayon)`. Affichez l'arbre indenté et le sous-arbre d'une racine.

CORRIGÉ

```
public enum Rayon {
    ALIMENTAIRE(null),
    MAISON(null),
    FRUITS(ALIMENTAIRE),
    LEGUMES(ALIMENTAIRE),
    BOISSONS(ALIMENTAIRE),
    ENTRETIEN(MAISON),
    CUISINE(MAISON),
    FRUITS_SECS(FRUITS),
    FRUITS_FRAIS(FRUITS);

    private final Rayon parent;

    Rayon(Rayon parent) { this.parent = parent; }

    public Rayon parent() { return parent; }

    public boolean estRacine() { return parent == null; }

    public int profondeur() {
        int d = 0;
        for (Rayon r = parent; r != null; r = r.parent) d++;
        return d;
    }

    public List<Rayon> chemin() {
        List<Rayon> c = new ArrayList<>();
        for (Rayon r = this; r != null; r = r.parent) c.add(0, r);
        return c;
    }

    public boolean sous(Rayon ancetre) {
        for (Rayon r = this; r != null; r = r.parent) if (r == ancetre) return true;
        return false;
    }
}
```

VÉRIFICATION

```
System.out.println(Rayon.FRUITS_FRAIS.chemin());
System.out.println(Rayon.FRUITS_FRAIS.profondeur());
System.out.println(Rayon.FRUITS_FRAIS.sous(Rayon.ALIMENTAIRE));
System.out.println(Arrays.stream(Rayon.values()).filter(r -> r.sous(Rayon.MAISON)).toList());
```

SORTIE ATTENDUE

```
[ALIMENTAIRE, FRUITS, FRUITS_FRAIS]
2
true
[MAISON, ENTRETIEN, CUISINE]
```

Barème : parents déclarés avant les enfants (4 pts) · `profondeur` et `chemin` corrects (6 pts) · `sous` remontant la chaîne (4 pts) · arbre indenté affiché (6 pts).

AFFICHAGE INDENTÉ (CORRIGÉ)

```
static void afficher(Rayon r, EnumMap<Rayon, List<Rayon>> enfants, String prefixe) {
    System.out.println(prefixe + r);
    for (Rayon e : enfants.getOrDefault(r, List.of())) afficher(e, enfants, prefixe + "  ");
}

EnumMap<Rayon, List<Rayon>> enfants = new EnumMap<>(Rayon.class);
for (Rayon r : Rayon.values())
    if (r.parent() != null) enfants.computeIfAbsent(r.parent(), k -> new ArrayList<>()).add(r);
for (Rayon r : Rayon.values()) if (r.estRacine()) afficher(r, enfants, "");
```

SORTIE ATTENDUE

```
ALIMENTAIRE
  FRUITS
    FRUITS_SECS
    FRUITS_FRAIS
  LEGUMES
  BOISSONS
MAISON
  ENTRETIEN
  CUISINE
```

10.5 Exercice 5 — Compter et regrouper (15 points)

Énoncé. À partir d'une liste de commandes `record Commande(Rayon rayon, int montant)`, produisez : le total par rayon (toutes les racines présentes, même à zéro), la commande la plus élevée par rayon, et le total général.

CORRIGÉ

```
record Commande(Rayon rayon, int montant) { }

List<Commande> commandes = List.of(
        new Commande(Rayon.FRUITS, 120),
        new Commande(Rayon.FRUITS_FRAIS, 80),
        new Commande(Rayon.BOISSONS, 45),
        new Commande(Rayon.CUISINE, 300));

// total par rayon, avec zéros
EnumMap<Rayon, Integer> totaux = new EnumMap<>(Rayon.class);
for (Rayon r : Rayon.values()) totaux.put(r, 0);
for (Commande c : commandes) totaux.merge(c.rayon(), c.montant(), Integer::sum);

// maximum par rayon
Map<Rayon, Optional<Commande>> max = commandes.stream()
        .collect(Collectors.groupingBy(Commande::rayon,
                () -> new EnumMap<>(Rayon.class),
                Collectors.maxBy(Comparator.comparingInt(Commande::montant))));

System.out.println(totaux);
System.out.println("total général : " +
totaux.values().stream().mapToInt(Integer::intValue).sum());
System.out.println(max);
```

SORTIE ATTENDUE

```
{ALIMENTAIRE=0, MAISON=0, FRUITS=120, LEGUMES=0, BOISSONS=45, ENTRETIEN=0, CUISINE=300, FRUITS_SECS=0,
FRUITS_FRAIS=80}
total général : 545
{FRUITS=Optional[Commande[rayon=FRUITS, montant=120]], BOISSONS=Optional[Commande[rayon=BOISSONS,
montant=45]], CUISINE=Optional[Commande[rayon=CUISINE, montant=300]],
FRUITS_FRAIS=Optional[Commande[rayon=FRUITS_FRAIS, montant=80]]}
```

Barème : `EnumMap` et non `HashMap` (4 pts) · préremplissage à zéro (3 pts) · `merge` correct (3 pts) · `groupingBy` avec fabrique `EnumMap` (3 pts) · total général (2 pts).

10.6 Exercice 6 — Un `switch` exhaustif (15 points)

Énoncé. Écrivez une méthode `String description(Statut)` couvrant les quatre statuts du chapitre 7.1, sous forme de `switch` expression sans `default`. Puis ajoutez un statut `ANNULE` à l'énumération : le code doit-il encore compiler ? Justifiez, et corrigez.

CORRIGÉ

```
static String description(Statut s) {
    return switch (s) {
        case OUVERT   -> "vient d'être créé";
        case EN_COURS -> "en traitement";
        case RESOLU   -> "traité, en attente de clôture";
        case FERME    -> "clos";
        case ANNULE   -> "annulé sans traitement";   // ajouté après l'erreur de compilation
    };
}
```

SORTIE ATTENDUE AVANT CORRECTION

```
error: the switch expression does not cover all possible input values
```

Réponse attendue à la question : non, le code ne compile plus — et c'est exactement l'intérêt. Le compilateur énumère les endroits à mettre à jour ; il suffit de suivre ses messages. Sans cette protection (avec un `default`), `ANNULE` aurait silencieusement pris une description fausse.

Barème : `switch` expression avec `->` (4 pts) · aucun `default` (4 pts) · ajout du cas `ANNULE` (3 pts) · justification correcte du rôle du compilateur (4 pts).

10.7 Exercice 7 — Persister sans se tromper (15 points)

Énoncé. Écrivez une classe `StockagePriorite` avec deux méthodes : `String encoder(Priorite)` et `Priorite decoder(String)`. L'encodage doit rester lisible par une version future de l'application qui aurait ajouté des constantes. Ajoutez un test qui échoue si une constante est renommée.

CORRIGÉ

```
public final class StockagePriorite {

    private StockagePriorite() { }

    public static String encoder(Priorite p) {
        return p.name();              // jamais ordinal()
    }

    public static Priorite decoder(String valeur) {
        try {
            return Priorite.valueOf(valeur);
        } catch (IllegalArgumentException e) {
            throw new IllegalArgumentException("Priorité inconnue en base : " + valeur, e);
        }
    }
}
```

TEST DE STABILITÉ

```
@Test
void les_noms_persistes_ne_doivent_pas_changer() {
    Set<String> attendus = Set.of("BASSE", "NORMALE", "HAUTE", "URGENTE");
    Set<String> reels =
Arrays.stream(Priorite.values()).map(Enum::name).collect(Collectors.toSet());
    assertEquals(attendus, reels, "Un renommage casse les données existantes : prévoir une
migration");
}
```

Barème : `name()` utilisé (5 pts) · exception explicite au décodage (3 pts) · test de stabilité complet (5 pts) · commentaire expliquant le risque (2 pts).

> **La variante à ne pas écrire**
>
> `return String.valueOf(p.ordinal());` — c'est court, ça marche, et ça corrompt les données au premier réordonnancement (chapitre 7.3, démonstration complète). Si vous trouvez ce code dans une base existante, prévoyez une migration immédiate : il est impossible de savoir, des mois plus tard, quelles lignes ont été écrites avec quel ordre.

10.8 Exercice 8 — Choisir la bonne structure (10 points)

Énoncé. Pour chacun de ces trois besoins, choisissez entre énumération simple, énumération avec champ `parent`, et hiérarchie `sealed` + `record`. Justifiez en deux phrases.

Cas	Besoin
a	Les codes d'erreur d'une API REST : `NOT_FOUND`, `CONFLICT`, `BAD_REQUEST`… chacun avec un code HTTP et un message par défaut.
b	Les événements d'un système de fichiers : création (nom), renommage (ancien nom, nouveau nom), suppression (nom, motif).
c	Le classement de produits du chapitre 5, avec ses rayons et sous-rayons.

Corrigé (une réponse argumentée vaut les points, pas la réponse exacte)

a) Énumération simple avec champs. Les cas ne diffèrent que par des valeurs (code, message), il n'y a pas de structure différente. Bonus : `EnumSet`/`EnumMap` disponibles pour indexer par code, et `valueOf` pour lire un paramètre d'URL.

b) Hiérarchie `sealed` + `record`. Chaque cas a des champs différents : un `record` par cas évite trois champs dont deux seraient `null`. Le `switch` exhaustif sur l'interface scellée donne la même sécurité qu'un switch d'énumération.

c) Énumération avec champ `parent`. Les cas sont de même forme (un nom, un parent), et la hiérarchie est une donnée à parcourir, pas un type à spécialiser. On garde `values()` et `EnumMap` pour les totaux par rayon.

Barème : bon choix pour chaque cas (1 pt chacun) · justification par la structure des données, pas par le goût (2 pts chacun) · une phrase sur ce qu'on perd avec l'autre choix (0,5 pt par cas).

ATELIER FINAL — 45 minutes, une énumération de bout en bout

Objectif : écrire une énumération complète, la tester, la trier et la persister, sans relire les corrigés.

1. **(8 min)** Choisissez un domaine (statuts de commande, niveaux de log, types de document). Écrivez l'énumération avec deux champs, un constructeur, deux accesseurs.
2. **(8 min)** Ajoutez `switch` expression exhaustif, sans `default`, pour produire un libellé d'affichage.
3. **(8 min)** Affichez trois tris : naturel, par nom, par attribut. Vérifiez à l'œil que les trois diffèrent, ou expliquez pourquoi ils coïncident.
4. **(8 min)** Ajoutez une hiérarchie par champ `parent` (deux niveaux suffisent), affichez-la indentée, filtrez un sous-arbre.
5. **(8 min)** Écrivez trois tests JUnit : stabilité des noms, ordre naturel, exhaustivité via `@EnumSource`.
6. **(5 min)** Ajoutez une constante : listez ce que le compilateur vous oblige à corriger. C'est votre checklist de livraison.

11 Sources et pour aller plus loin

> Toutes les sorties de ce document ont été produites sur Debian 13 avec Temurin 25.0.5 en novembre 2026. Les références ci-dessous sont les sources primaires : c'est vers elles qu'il faut revenir quand une version change.

11.1 Spécification et API

Document	Ce qu'on y trouve, et le chapitre concerné
`docs.oracle.com/javase/specs/jls/se25/html/jls-8.html#jls-8.9`	JLS §8.9 « Enum Classes » : la définition normative, les membres implicitement déclarés, le constructeur privé, la sérialisation. Chapitres 1 et 2.
`docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/Enum.html`	`name()`, `ordinal()`, `compareTo`, `valueOf`, et les recommandations d'usage (`toString` plutôt que `name` pour l'affichage). Chapitres 1.3 et 3.2.
`…/java.base/java/lang/Comparable.html`	Le contrat de `compareTo` : antisymétrie, transitivité, cohérence avec `equals`. Chapitre 3.
`…/java.base/java/util/Comparator.html`	`comparing`, `thenComparing`, `reversed`, `naturalOrder`, `nullsFirst`/`nullsLast`, et l'avertissement sur la cohérence avec `equals`. Chapitres 4.2 et 8.2.
`…/java.base/java/util/EnumSet.html`	`allOf`, `range`, `complementOf`, `copyOf` ; ordre d'itération garanti. Chapitres 4.5 et 7.2.
`…/java.base/java/util/EnumMap.html`	Ordre garanti sur `keySet()`, `values()`, `entrySet()` ; performance et usage. Chapitres 4.5 et 7.2.
`…/java.base/java/text/Collator.html`	Tri linguistique : forces PRIMARY/SECONDARY/TERTIARY, `getInstance(Locale)`. Chapitre 4.4.
`…/java.base/java/lang/String.html#compareTo(java.lang.String)`	Définition du tri lexicographique sur les valeurs de `char` : c'est la cause des surprises sur les nombres et les accents. Chapitre 4.4.
`…/java.base/java/util/stream/Collectors.html`	`groupingBy` avec fabrique de `Map`, `counting`, `maxBy`. Chapitres 7.2 et 10.5.
`….docs/api/java.base/java/lang/reflect/Constructor.html#newInstance`	Mention du refus d'instanciation d'un enum (`Cannot reflectively create enum objects`). Chapitre 1.4.

11.2 Code source du JDK : la meilleure documentation

Le JDK livré contient ses propres sources dans `$JAVA_HOME/lib/src.zip`. C'est la façon la plus rapide de vérifier un comportement — et c'est ce qui a servi à citer l'implémentation de `compareTo` dans ce document.

COMMANDE — RETROUVER ET LIRE UNE CLASSE DU JDK DANS `SRC.ZIP`

```
# où se trouve le code source des classes de la bibliothèque standard ?
echo "$JAVA_HOME/lib/src.zip"

# lire l'implémentation de compareTo dans java.lang.Enum
unzip -p "$JAVA_HOME/lib/src.zip" java.base/java/lang/Enum.java | grep -n -A7 "public final int compareTo"
```

EXTRAIT DU FICHIER `JAVA.BASE/JAVA/LANG/ENUM.JAVA` DU JDK 25, LIGNES 215 À 221

```
    public final int compareTo(E o) {
        Enum<?> other = o;
        Enum<E> self = this;
        if (self.getClass() != other.getClass() && // optimization
            self.getDeclaringClass() != other.getDeclaringClass())
            throw new ClassCastException();
        return self.ordinal - other.ordinal;
    }
```

Ce fichier est la référence absolue quand la Javadoc reste allusive : elle décrit le contrat, le code source décrit le comportement. Prenez l'habitude de l'ouvrir — c'est aussi la meilleure façon d'apprendre les idiomes de la bibliothèque standard.

11.3 Pour aller plus loin

Sujet	Par où commencer
Pattern matching complet	Les records patterns et les gardes `when` (Java 21+), les patterns de constantes dans un `switch`. Chapitre 6.3 comme point de départ.
Classes scellées	JLS §8.1.6 « Permitted Direct Subclasses » et le guide `sealed`/`record`. Chapitre 5.4.
Sérialisation et compatibilité	Spécification de sérialisation Java, section sur les énumérations : la représentation est le nom, non modifiable. Chapitre 7.3.
Normalisation Unicode	`java.text.Normalizer` : deux chaînes visuellement identiques peuvent différer octet par octet (`É` composé ou décomposé). À connaître si vous lisez des noms depuis un fichier externe.
Tests paramétrés	`@EnumSource`, `@CsvSource`, `@MethodSource` dans le guide utilisateur JUnit 5. Chapitre 7.5.
Performance des collections	`EnumSet`/`EnumMap` dans la Javadoc : implémentations `RegularEnumSet` et `JumboEnumSet` selon le nombre de constantes (seuil de 64). Chapitre 4.5.

La réponse en une phrase aux deux questions du départ

« Peut-on trier par le nom automatiquement ? » Oui, avec `Comparator.comparing(E::name)` — mais ce n'est jamais l'ordre par défaut, et l'ordre alphabétique pur réserve deux surprises : les nombres et les accents.
« Peut-on créer une hiérarchie automatique par le nom ? » Non. Java n'offre aucune hiérarchie implicite entre constantes ; la seule « automatique » est l'ordre linéaire de déclaration. Pour un arbre, un champ `parent` explicite (ou une hiérarchie `sealed` + `record` quand les cas diffèrent par leur structure) est la seule solution fiable — et elle tient en dix lignes.